§2.1 逻辑电路描述

一、框图：

①用**方框、圆框**等粗略表示系统的输入、输出和功能模块（或称子系统）；

②用**带箭头连线**表示模块之间主要信息通路、其流向，可能还包含控制信号。

系统总框图：表示了一个完整的系统模块组成。

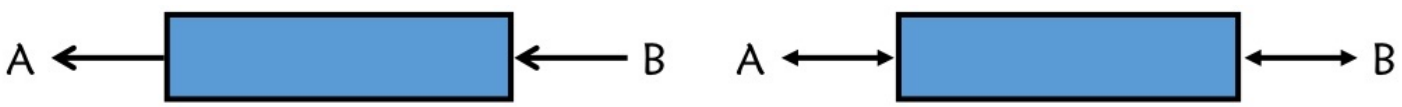
子系统框图：将一个大系统划分成几个子系统，通过这几个子系统的说明构成整个系统的总框图。即 由子系统构成其框图。

二、电路图：

1. 图面布局及总线(*Drawing Layout and Buses*)

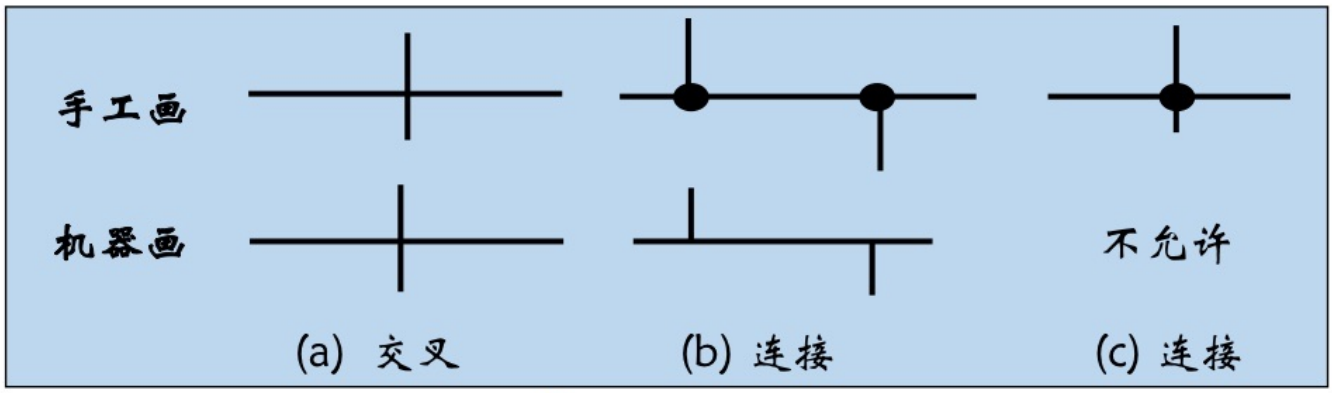
原理图将更注重描述原理过程所需要使用的标注。在示例性**逻辑图及原理图**中，一般约定：

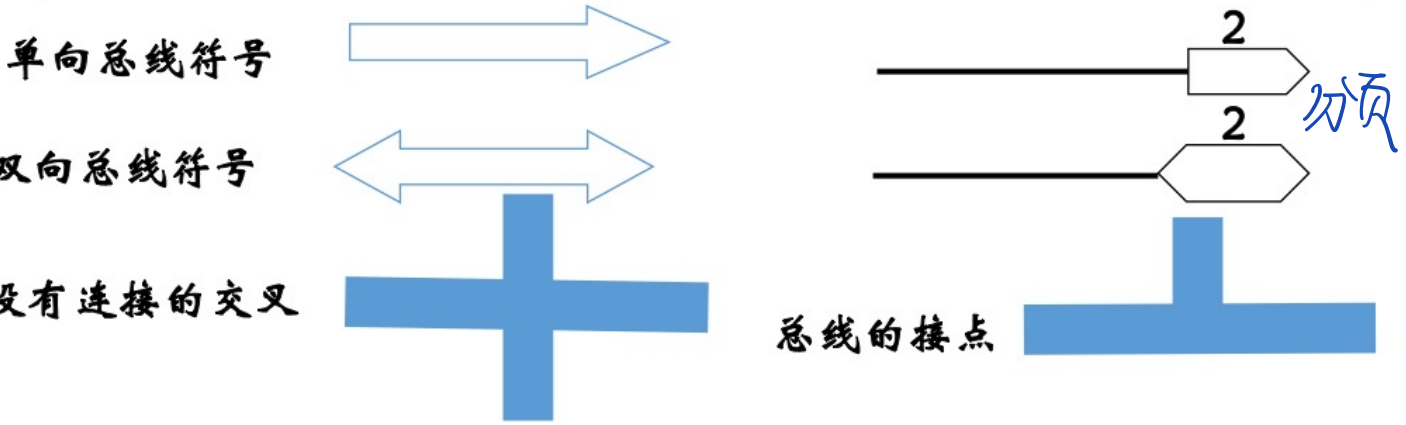
- **信息流**：默认从左至右，或者从上到下，若不能保证则使用**箭头**提示信息流方向，有单向和双向之分。



- **逻辑器件**：**输入端**画在左边，**输出端**画在右边。
- **图中应注明**所用集成电路的**型号**、连线的**引端（脚）号**、在整个逻辑图中的**编号**以及输入、输出**信号名**等。
- **需要分页画出的逻辑图**要合理地划分出每页的模块，既要完整又要使页与页之间的连线尽可能地少，并清楚地标注出它们之间的连接关系。
- **信息线的交叉点**：手工作图时用圆点表示，CAD软件作图时用T型连接。参见下图所示。

- 总线的表示法





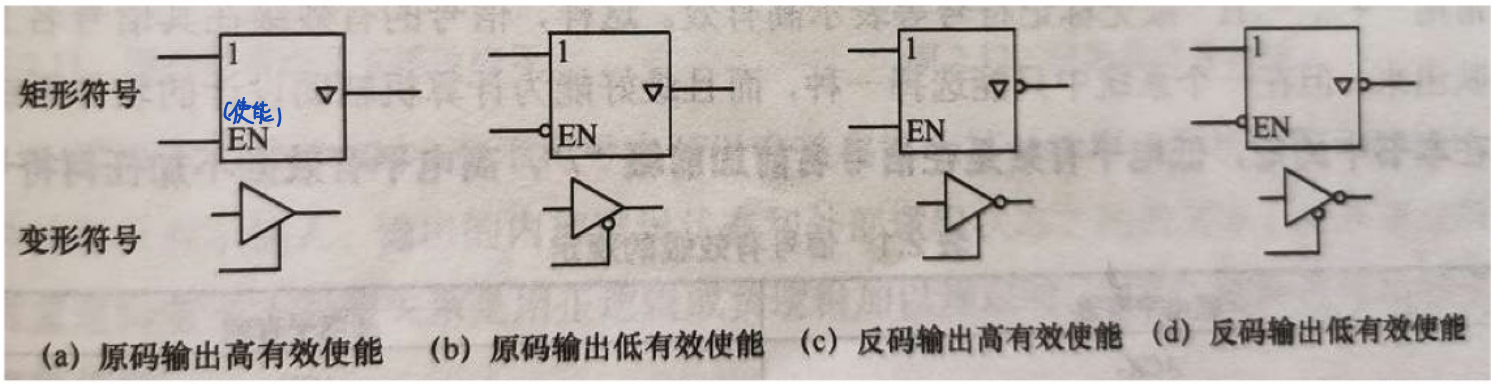
2. 逻辑门符号标准：

长方形 / 变形符号

等效符号（输入取反）：输入信号进行有效级变换（变反），并按照DeMorgan定律得到

电路名称	方框型符号	变形符号
跟随器 √	1	
非 门	1	
与 门	&	
或 门	≥1	
与非门	&	
或非门	≥	
与或非门	& ≥	
异或门	=1	

三态门符号（三态缓冲器/驱动器）



逻辑门的等效符号

等效符号和原符号如何区分？

电路名称	原符号	等效符号 √
跟随器	1	1
非　门	1	1
与　门	&	≥1
或　门	≥1	&
与非门	&	≥1
或非门	≥1	&

3）信号名和有效级

➢信号（逻辑变量）命名

为了电路分析和设计，对电路的各**输入、输出信号**进行**规范化命名**，即取名最好源自信号的名称或者是它的英文缩写。如：

数据信号$Data_i$ (D_i)；
地址信号$Addr_i$ (A_i)；
控制信号$Cont_i$ (C_i)、Reset、Set、 … ；
检测信号Ready、Error、 …、
等待信号Wait、…；
片选信号CS；使能信号EN； …。

➢ 信号的有效级（Active levels for Signals）

① 控制信号、测试信号等：

此类信号都有一个与之对应的有效级。

当信号处在其有效级时，逻辑电路才能正确地执行其功能。

② 在正逻辑规定中，高电平与逻辑状态“1”等效；
低电平与逻辑状态“0”等效。

③ 有效级分高有效或低有效。

高有效：信号为高电平或为逻辑状态“1”时为逻辑电路功能有效；

低有效：信号为低电平或为逻辑状态“0”时为逻辑电路功能有效。

④ 信号有效级的约定（即表示法）

用特殊符号作为信号名的前缀或后缀，这些符号反映了信号的有效性，如下表所示。

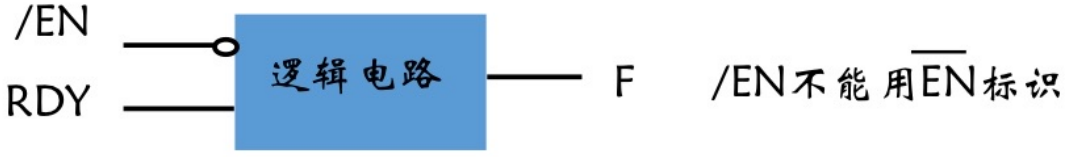
低电平有效	高电平有效
ACK-	ACK+
ERROR.L	ERROR.H
ACS(L)	ACS(H)
CS*	CS
/EN	EN
RESET#	RESET

本书采用表中“EN(高有效)、/EN(低有效)”的这一组表示法，特殊符号为正斜杠“/”。

⑤ 信号名不能采用反变量符号，不能采用表达式（本教材约定）。

如下图中，RDY（准备好）为测试信号，/EN（使能）为控制信号，

当RDY为高电平、/EN为低电平时，则该电路工作，即F为有效。



4）引端（通俗一点叫引脚）的有效级(*Active levels for pins*)

●**引端的有效级：**是指电路的输入、输出上的物理量与电路内部逻辑状态的对应关系。

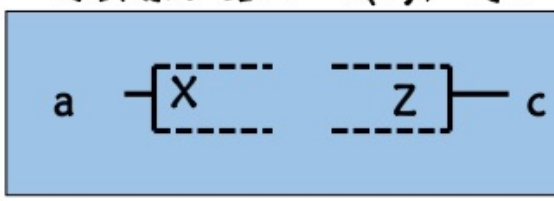
●要求：选用器件的**引端的有效级与所给信号的有效级要相匹配**。本书采用“逻辑非符号体制”（小圆圈表示逻辑非），不采用“极性符号体制”。

● **逻辑非符号体制**在本体制下存在两级对应关系：

① **首先是电路的外部逻辑状态与内部逻辑状态的对应关系。**

下例：所示框图中a、c为外部逻辑状态符号；X、Z为内部逻辑状态符号。

- 器件框图上不带逻辑非符号（即小圆圈）的输入a与X、输出c与Z的关系：若Z = f(X)，则c = f(a)

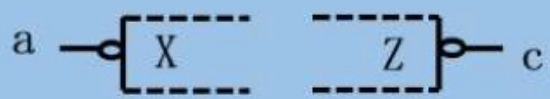


a	X
0	0
1	1

Z	c
0	0
1	1

- 器件框图上带逻辑非符号的输入a与X、输出c与Z的关系：

若$Z = f(X)$，则 $c = \overline{Z} = \overline{f(X)} = f(\overline{a})$



a	X
0	1
1	0

Z	c
0	1
1	0

② **电路的输入、输出信号的物理量与电路的外部逻辑状态的对应关系，用正逻辑或负逻辑加以约定。**

- **正逻辑约定**

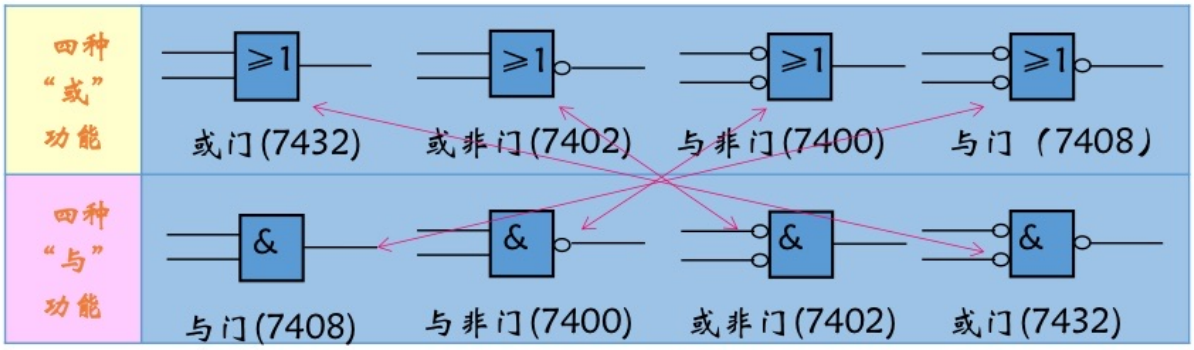
逻辑电平对应关系：H（高电平）—— “1”（外部逻辑状态）

L（低电平）—— “0”（外部逻辑状态）

5）引端有效级的变换

- 目的：使逻辑电路的功能一目了然；
- 结果：使所选用器件（集成电路）引端的有效级与所给的信号有效级相匹配；
- 方法：信号与引脚端不匹配时，对器件引端的有效级进行变换。

下面两组、每组四种分别表示四个可以完成同一逻辑功能的器件：

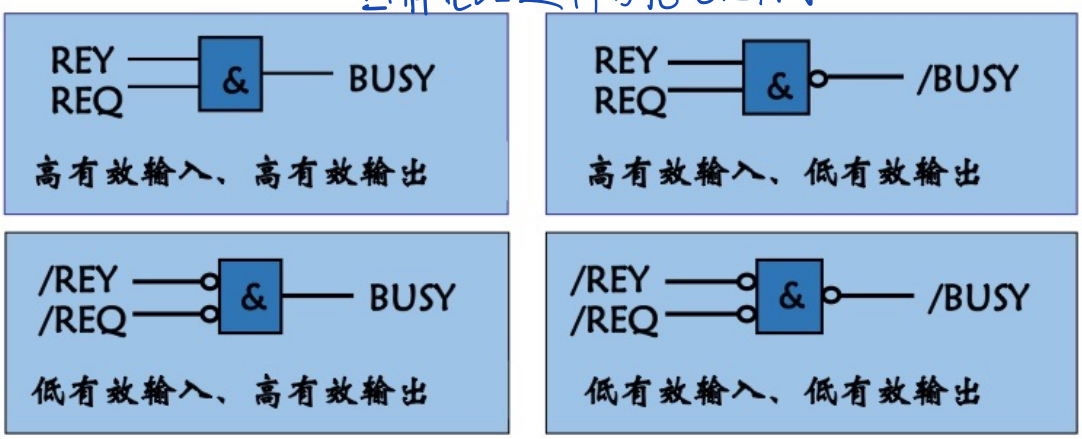


● 引端有效级的变换，包括：

输入引端和输出引端的变换：或为高有效、或为低有效，

例如，下图给出了实现 Busy = REY · REQ 逻辑功能对应的四种电路。

四种电路逻辑功能完全相同



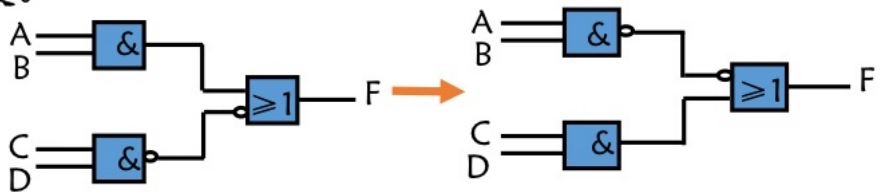
● 引端有效级的变换规则

在保持输入与输出逻辑功能不变的条件下，可依如下变换规则对逻辑图进行任意变换。

规则1：任何模块电路输入或输出端加上或删去逻辑非符号（即小圆圈），且其对应的信号有效级变反，则逻辑图的功能不变。



规则2：逻辑电路图内部连线的两端，同时加上或删去逻辑非符号，则逻辑图的功能不变。



规则3：单个逻辑非符号在电路内部连线两端移动时逻辑图的功能不变。



规则4：若一个门的输入输出端同时加上或删去逻辑非符号，或输入、输出信号有效级同时取反，且门的符号“与”、“或”互变时，则得到的新的逻辑图的功能不变。



即：同时改变有效级或逻辑非，或改变两处逻辑非

变换目标：

(1) 器件的输出信号有效级应与对应输出引端的有效级一致。即输出端有逻辑非符号，输出信号为低有效，否则为高有效。

(2) 若输入信号有效级与其对应的输入端有效级一致时，当该信号有效时，则器件内部逻辑功能有效。

(3) 若输入信号有效级与其对应的输入端有效级不一致时，则当该信号无效时，则器件内部逻辑功能才有效。这是应尽量避免出现的情况。

3. 时间图 (Timing Diagram：时序图)：

时间图是任何数字系统设计文档中的一个重要部分。是分析、设计和调试电路的重要辅助手段，是数字系统中的重要文档之一。

时间图是描述数字电路中信号随时间变化的一种图形。反映逻辑信号之间对应的时间关系，特别是要能反映出关键信号之间因果关系。时间图可用来说明系统内信号之间的时间关系，同时，它又用于定义对外部信号的时间要求。与时序图相关的时间参数：有信号传输转换时间（t_{TLH}和t_{THL}）、信号传输延迟时间（t_{PLH}和t_{PHL}）。

- 时间图中最重要的是信号变化之间的时间延迟特性。 延迟时间是指相关的两个变化中心之间的时间间隔。每种延迟通常给出一个范围值。 即（最小、典型、最大）时间值
- 总线上的所有信号具有相同的延迟，并不需要表示其中任一位信号在瞬时的具体逻辑值，仅需标注这一组信号发生转换的时间，尤其重要的是要标注发生的转换相对于系统中控制信号发生转换时的相对时间差。
- 在数字系统中，信号间的相对时间关系是非常重要的，达到精确的时间配合才能实现预定的逻辑功能。

4.结构化逻辑描述 (Structured Logic Description)

说明结构化逻辑器件的内部功能，如PLA、存储器芯片或者某些具有专门功能的中、大规模器件；
用逻辑代数等式、状态表(图)、功能表或程序表等形式说明。（在以后的分析和设计中再介绍）

5.电路说明 (Circuit Description)：

用文字简明叙述电路的使用方法；
解释内部的工作方法；
列出设计和操作中所有可能的潜在缺陷，以及在使用不当时隐含的问题。
（在以后的分析和设计中再介绍）

§2.2 组合逻辑电路分析与设计

一、分析：

分析过程：①仔细观察逻辑电路图；②列出逻辑表达式，用代数法或卡诺图对其化简；③列出真值表或简化真值表，分析出电路实现的功能；④用文字描述电路实现的功能；⑤对电路的性能进行评价，必要时提出改进方案。

逻辑电路分为组合逻辑电路和时序逻辑电路两类。组合电路是不带反馈的电路，其特点为：电路输出仅取决于当时（前）的输入，而与过去的输入情况无关。

1）电平推导法（根据输出推出输入的电平）

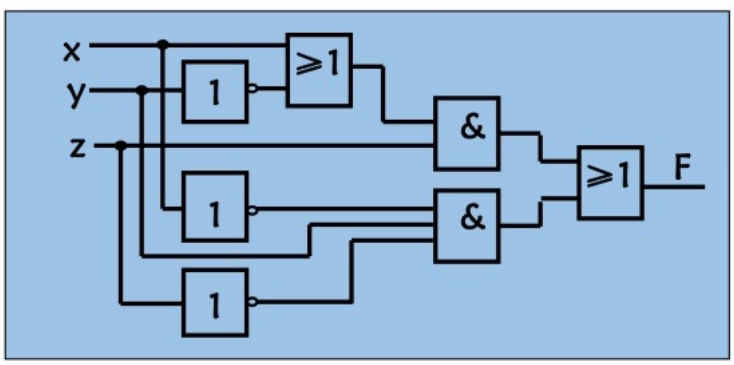
逐级电平推导法：先假定输出为某一电平值（高低电平用“1或0”来标识），然后，反向逐级向前推导，直到推得输入的电平值。

2）穷举法（从输入推输出）

穷举法的结果是真值表。即列出n个输入变量的所有2^n个输入组合，并根据每一个输入组合决定所有门的输出，逐级推出电路的输出，得到真值表。

例：分析如图三输入一输出的逻辑电路。

x	y	z	F
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1



3）逻辑代数法

根据电路逐级写出各门的输出表达式，直至写出整个电路的输出逻辑表达式。例：　$F = (x+\bar{y})z + (\bar{x}y\bar{z})$

根据布尔代数进行表达式变换，如下：

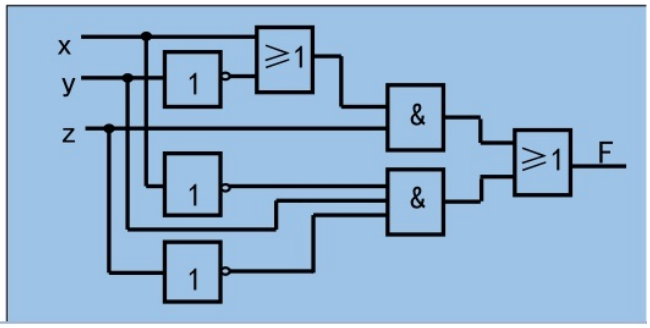
$F = xz + \bar{y}z + \bar{x}y\bar{z}$　　与或式

$= (\bar{x} + z)(y + z)(x + \bar{y} + \bar{z})$　　或与式

$= \overline{\overline{(xz)}\ \overline{(\bar{y}z)}\ \overline{(\bar{x}y\bar{z})}}$

与非—与非式

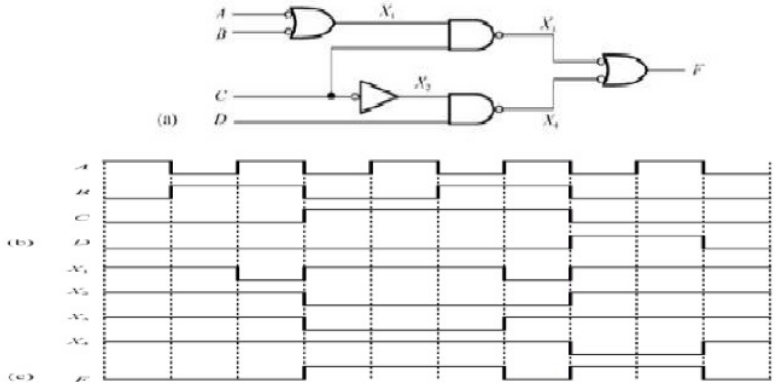
上述表达式对应不同结构的逻辑电路。



4）数字波形图分析法

这种方法是对逻辑门的所有输入变量施以输入波形，逐级画出各个门电路的输出波形，乃至画出最后的输出波形。

例，图（a）所示的逻辑电路有A、B、C、D四个变量，输入波形如图（b）所示。



二、设计：

目的：依逻辑功能，利用逻辑代数方法实现电路设计的一般步骤如下所示：

分析设计要求	确定输入、输出的变量；逻辑关系；有无d(无关项)
↓	
列出真值表	填入卡诺图进行化简
↓	
写出最简逻辑表达式	由卡诺图得到最简与或式
↓	
表达式变换	根据所选用门的类型；输入无反变量；有效级等。
↓	
画出电路逻辑图	

考虑设计约束

要求（设计优化目标）：电路用最少的逻辑门（集成块）、最少的输入端数。

1. 根据问题写出逻辑表达式：

（1）逻辑问题描述—真值表—卡诺图化简—逻辑表达式

例　设计一个二进制一位全加器。→有前后进位

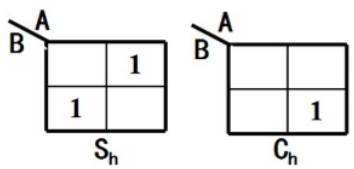
➢ **半加器** *Half-Adder*（仅用与非门）

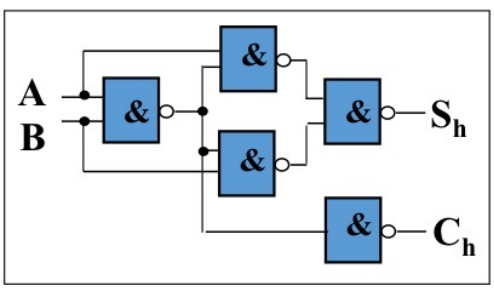
输入变量：加数A、被加数 B

输出函数：相加和S_h、进位C_h

$$S_h = \overline{A}B + A\overline{B} = A \oplus B \qquad C_h = AB$$
$$= A\overline{AB} + B\overline{AB} \qquad = \overline{\overline{AB}}$$

A B	S_h C_h
0 0	0 0
0 1	1 0
1 0	1 0
1 1	0 1



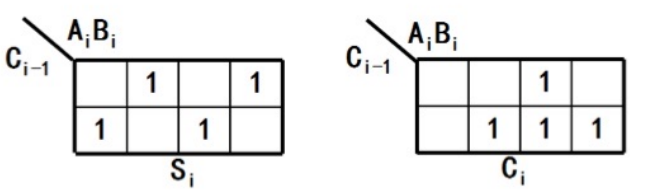


➢ **全加器** *Full-Adder*

输入变量：加数 A_i、被加数 B_i 、来自低位的进位 C_{i-1}

输出函数：本位和 S_i、本位向高位的进位C_i

A_i	B_i	C_{i-1}	S_i	C_i
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1



$$S_i = A_i \oplus B_i \oplus C_{i-1}$$
$$= \overline{A}_i\overline{B}_i C_{i-1} + \overline{A}_i B_i \overline{C}_{i-1} + A_i\overline{B}_i\overline{C}_{i-1} + A_i B_i C_{i-1}$$

$$C_i = A_iB_i + A_i C_{i-1} + B_i C_{i-1}$$

可以用二级“与或”电路实现此加法器

用“与或非”门实现全加器

$$S_i = A_i \oplus B_i \oplus C_{i-1}$$
$$= \overline{A_i}\overline{B_i}\,C_{i-1} + \overline{A_i}B_i\overline{C_{i-1}} + A_i\overline{B_i}\overline{C_{i-1}} + A_iB_i\,C_{i-1}$$

C_{i-1} \ A_iB_i	00	01	11	10
0		1		1
1	1		1	

S_i

$$C_i = A_iB_i + A_i\,C_{i-1} + B_i\,C_{i-1}$$

C_{i-1} \ A_iB_i	00	01	11	10
0			1	
1		1	1	1

C_i

$$\overline{C_i} = \overline{A_iB_i + A_i\,C_{i-1} + B_i\,C_{i-1}}$$

$$S_i = m_1 + m_2 + m_4 + m_7$$ 为了使用原变量

C_{i-1} \ A_iB_i	00	01	11	10
0	1	1		1
1	1			

$\overline{C_i}$

$m_1 = C_{i-1} \cdot \overline{C_i}$
$m_2 = B_i \cdot \overline{C_i}$
$m_4 = A_i \cdot \overline{C_i}$
$m_7 = A_i \cdot B_i \cdot C_{i-1}$

} 禁止法

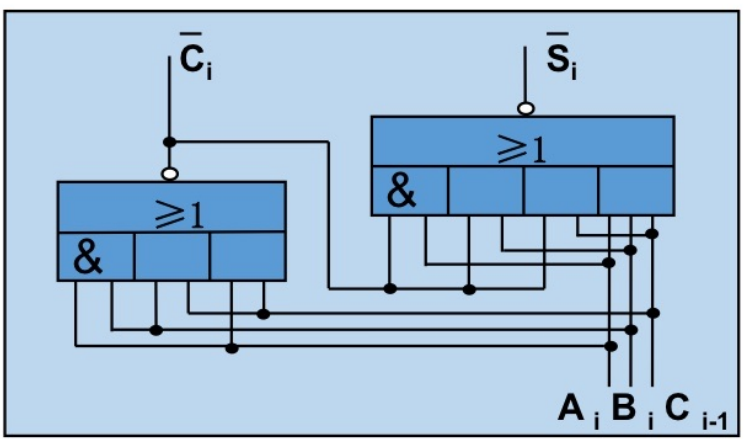
故 $S_i = C_{i-1} \cdot \overline{C_i} + B_i \cdot \overline{C_i} + A_i \cdot \overline{C_i} + A_i \cdot B_i \cdot C_{i-1}$

$= \overline{C_i} \cdot (C_{i-1} + B_i + A_i) + A_i \cdot B_i \cdot C_{i-1}$

用“与或非”门实现全加器

$$\overline{C_i} = \overline{A_iB_i + A_i\,C_{i-1} + B_i\,C_{i-1}}$$

$$\overline{S_i} = \overline{C_i} \cdot (C_{i-1} + B_i + A_i) + A_i \cdot B_i \cdot C_{i-1}$$



此电路没有使用非门，
但两个输出都是低有效！

(2) 逻辑问题描述—简化真值表（考虑无关项）—逻辑表达式

➢ 设计一个2位2进制正整数比较大小的电路－比较器 *Comparators*

输入变量：两个正整数 $x = x_2x_1$， $y = y_2y_1$

输出函数：三个比较结果 $F_1(x > y)$，$F_2(x < y)$，$F_3(x = y)$

①根据先比较高位后比较低位的原则，列出使各函数为值为1的简化真值表：

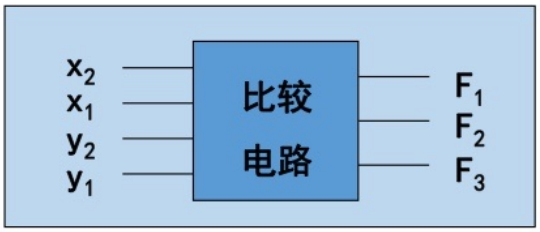
x_2	y_2	x_1	y_1	F_1	F_2	F_3
1	0	d	d	1	0	0
0	1	d	d	0	1	0
0	0	1	0	1	0	0
		0	1	0	1	0
		0	0	0	0	1
		1	1	0	0	1
1	1	1	0	1	0	0
		0	1	0	1	0
		0	0	0	0	1
		1	1	0	0	1

②由简化真值表**直接写出逻辑表达式**

$F_1 = x_2\overline{y_2} + \overline{x_2}\,\overline{y_2}x_1\overline{y_1} + x_2y_2x_1\overline{y_1}$

$F_2 = \overline{x_2}y_2 + \overline{x_2}\,\overline{y_2}\,\overline{x_1}y_1 + x_2y_2\overline{x_1}y_1$

$F_3 = \overline{x_2}\,\overline{y_2}\,\overline{x_1}\,\overline{y_1} + \overline{x_2}\,\overline{y_2}x_1y_1 + x_2y_2\overline{x_1}\,\overline{y_1} + x_2y_2x_1y_1$



x_2	y_2	x_1	y_1	F_1	F_2	F_3
1	0	d	d	1	0	0
0	1	d	d	0	1	0
0	0	1	0	1	0	0
		0	1	0	1	0
		0	0	0	0	1
		1	1	0	0	1
1	1	1	0	1	0	0
		0	1	0	1	0
		0	0	0	0	1
		1	1	0	0	1

逻辑电路图参见书P58图2.28（c）。

设计一个一位十进制数等值比较器？

（3）逻辑问题描述—逻辑表达式

由逻辑问题描述直接写出逻辑表达式。

例　设计一个房间报警电路，如果：

①意外事件发生输入信号PANIC为逻辑状态1；

②使能输入信号ENABLE为1、出口标志输入信号EXITING为逻辑状态0、房间没有加密信号(SECURE)；则报警输出信号ALARM为1。

如果：窗(WINDOW)、门(DOOR)及车库(GARAGE)都是逻辑状态1

则房间加密信号(SECURE)有效。

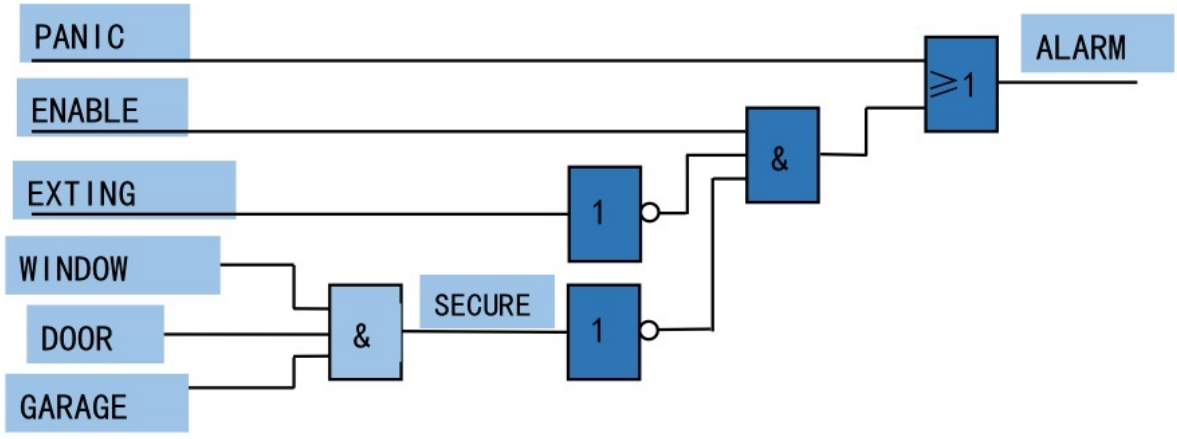
$$ALARM = PANIC + ENABLE \cdot \overline{EXITING} \cdot \overline{SECURE}$$

$$SECURE = WINDOW \cdot DOOR \cdot GARAGE$$

$$ALARM = PANIC + ENABLE \cdot \overline{EXITING} \cdot \overline{(WINDOW \cdot DOOR \cdot GARAGE)}$$

报警电路逻辑图

$$ALARM = PANIC + ENABLE \cdot \overline{EXITING} \cdot \overline{(WINDOW \cdot DOOR \cdot GARAGE)}$$



2. 逻辑电路的变换：

为了提高电路的速度，提高器件的利用率，从而减少IC的数量、也为了减少外部的连接线和提高电路的可靠性，需要对从逻辑表达式直接画出的逻辑电路图进行变换，尽可能使其用同一类型的输出端带非的门来实现。

(1) “与—或”电路变换为“与非—与非”电路

例 $F = \overline{\overline{A + BC + DEF + G}}$

$= A + BC + DEF + G$ （原函数二次求反）

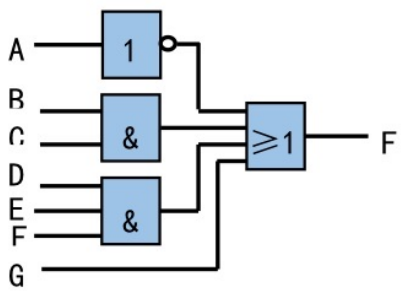
$= A \cdot \overline{BC} \cdot \overline{DEF} \cdot \overline{G}$ （运用反演规则）

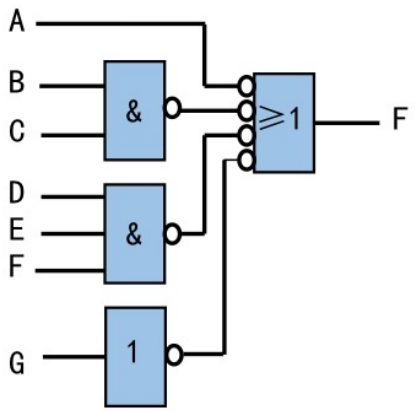
$= A \cdot \overline{\overline{BC} \cdot \overline{DEF} \cdot \overline{G}}$

对应的二个不同的电路如下：

$F = \overline{A + BC + DEF + G}$

$= A \cdot \overline{BC} \cdot \overline{DEF} \cdot \overline{G}$





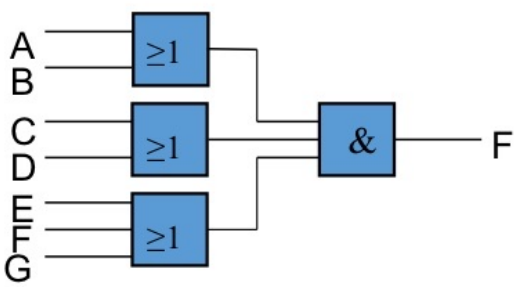
(2) “或—与”电路变换为“或非—或非”电路

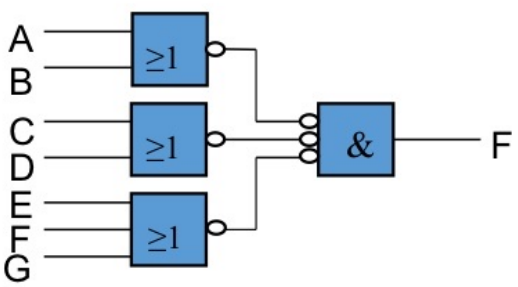
例 $F = (A + B)(C + D)(E + F + G)$

$= \overline{\overline{(A + B)(C + D)(E + F + G)}}$ （原函数二次求反）

$= \overline{(A + B)} + \overline{(C + D)} + \overline{(E + F + G)}$ （运用反演规则）

$= \overline{(A + B)} \cdot \overline{(C + D)} \cdot \overline{(E + F + G)}$ （运用反演规则）





（3）“与—或”电路变换为“与或非”电路

例 $F = \overline{\overline{AC} + \overline{AB}}$ (a)

$= \overline{\overline{AC}}\,\overline{\overline{AB}}$ （原函数二次求反） (b)

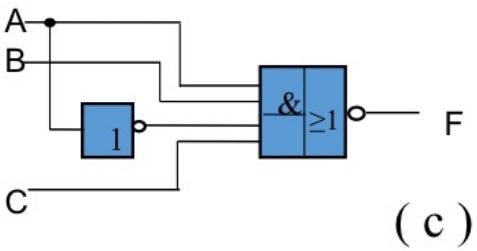
$\overline{F} = \overline{\overline{AC}}\,\overline{\overline{AB}}$

（运用反演规则）

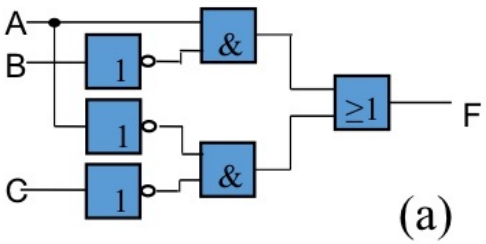
$= (A + C)(\overline{A} + B)$

$= AB + \overline{A}\overline{C}$

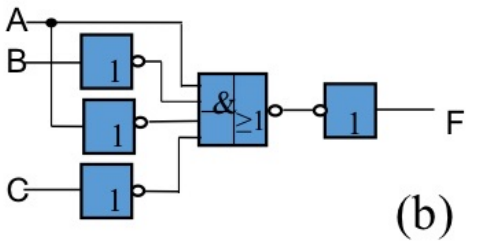
$F = AB + \overline{A}\overline{C}$ (c)



(c)



(a)



(b)

（4）减少集成块的数量

Reduce the Numbers of IC

小规模门电路SSI是把几个相同的门封装在同一个集成块中，在逻辑电路中使用的SSI的数目越少，则电路的印刷电路扳的面积、功耗、总成本越小，而可靠性越高。

所以，减少SSI的数目是化简的最终目标。

在实际应用中，对SSI的计算与分立元件的计算不一样。

三、题目：

组合逻辑电路分析：

分析过程：①仔细观察逻辑电路图；②列出逻辑表达式，用代数法或卡若图对其化简；③列出真值表或简化真值表，分析出电路实现的功能；④用文字描述电路实现的功能；⑤对电路的性能进行评价，必要时提出改进方案。

分析方法：电平推导法（输入—>输出）；穷举法（真值表，输出—>输入）；逻辑代数法；波形图法。

组合逻辑电路设计：

设计过程：①分析设计要求（确定输入、输出变量，逻辑关系，有无d）；②列出真值表式，卡若图对其化简；③写出最简逻辑表达式，卡诺图得到最简与或式；④表达式变换，根据选用门类型、输入无反变量、有效级等；⑤画出电路逻辑图，最少逻辑门或IC块、最少输入端数。

1.分析

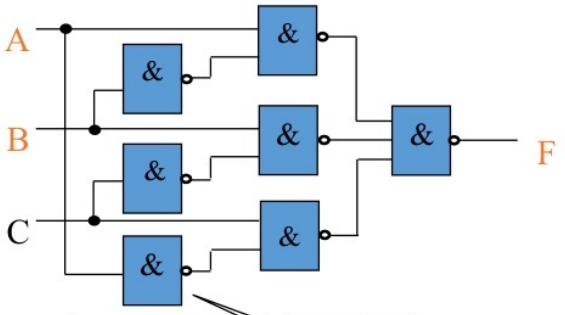
分析例1：分析如图逻辑电路。

写出最简表达式：

$$F = \overline{\overline{A\overline{B}}\ \overline{B\overline{C}}\ \overline{C\overline{A}}}$$

$$= A\overline{B} + B\overline{C} + C\overline{A}$$

从表达式直接看不出明确的逻辑关系，再通过真值表来分析：



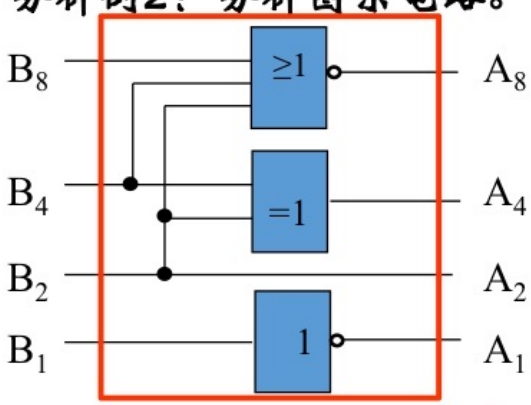
A B C	F
0 0 0	0
0 0 1	1
0 1 0	1
0 1 1	1
1 0 0	1
1 0 1	1
1 1 0	1
1 1 1	0

有三个门表示不规范

从真值表可以得出

这是一个判三变量非一致（不同）电路。

分析例2：分析图示电路。



写出最简表达式：

$$A_8 = \overline{B_8 + B_4 + B_2} = \overline{B}_8\overline{B}_4\overline{B}_2$$

$$A_4 = B_4 \oplus B_2 = \overline{B}_4 B_2 + B_4\overline{B}_2$$

$$A_2 = B_2$$

$$A_1 = \overline{B}_1$$

这是一个BCD码对9变补器。

$B_8\ B_4\ B_2\ B_1$	$A_8\ A_4\ A_2\ A_1$
0 0 0 0	1 0 0 1
0 0 0 1	1 0 0 0
0 0 1 0	0 1 1 1
0 0 1 1	0 1 1 0
0 1 0 0	0 1 0 1
0 1 0 1	0 1 0 0
0 1 1 0	0 0 1 1
0 1 1 1	0 0 1 0
1 0 0 0	0 0 0 1
1 0 0 1	0 0 0 0
1 0 1 0	0 1 1 1
1 0 1 1	0 1 1 0
1 1 0 0	0 1 0 1
1 1 0 1	0 1 0 0
1 1 1 0	0 0 1 1
1 1 1 1	0 0 1 0

分析例3：分析如图逻辑电路。

1. 写出最简表达式：

$y_0 = x_0$

$y_1 = x_1 \oplus x_0$

$y_2 = x_2 \oplus (x_1 + y_1)$

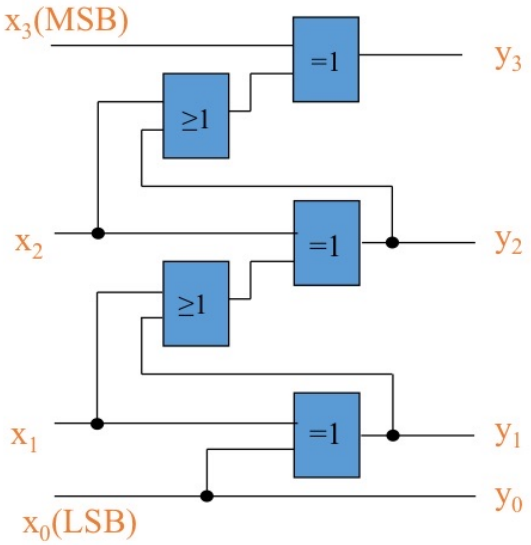
$= x_2 \oplus (x_1 + x_1 \oplus x_0)$

$= x_2 \oplus (x_1 + x_0)$

$y_3 = x_3 \oplus (x_2 + y_2)$

$= x_3 \oplus (x_2 + x_2 \oplus (x_1 + x_0))$

$= x_3 \oplus (x_2 + x_1 + x_0)$



LSB: Least Significant Bit

MSB: Most Significant Bit

TONG UNIVERSITY

2. 函数最简表达式：

$y_0 = x_0$

$y_1 = x_1 \oplus x_0$

$y_2 = x_2 \oplus (x_1 + x_0)$

$y_3 = x_3 \oplus (x_2 + x_1 + x_0)$

3. 通过真值表来分析：

$x_3x_2x_1x_0$	$y_3y_2y_1y_0$
0 0 0 0	0 0 0 0
0 0 0 1	1 1 1 1
0 0 1 0	1 1 1 0
0 0 1 1	1 1 0 1
0 1 0 0	1 1 0 0
0 1 0 1	1 0 1 1
0 1 1 0	1 0 1 0
0 1 1 1	1 0 0 1
1 0 0 0	1 0 0 0
1 0 0 1	0 1 1 1
1 0 1 0	0 1 1 0
1 0 1 1	0 1 0 1
1 1 0 0	0 1 0 0
1 1 0 1	0 0 1 1
1 1 1 0	0 0 1 0
1 1 1 1	0 0 0 1

结论：

这是一个四位二进制变补器，也称16变补器。

已知：$x = x_{n-1}\, x_{n-2} \cdots x_1\, x_0$

则：x 求补为：

$\overline{x}_{n-1}\, \overline{x}_{n-2} \cdots \overline{x}_1\, \overline{x}_0 + 1$

$= y_{n-1}\, y_{n-2} \cdots y_1\, y_0$

变补的规律是：

从数值位的最低位开始进行逻辑转换！

$y_i = x_i \oplus (x_{i-1} + y_{i-1})$

$y_i = x_i \oplus (x_{i-1} + x_{i-2} \cdots + x_1 + x_0)$

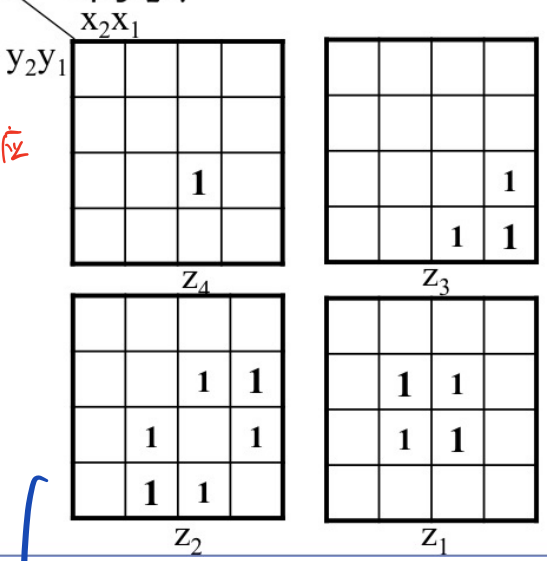
2.设计

设计例1： 设计一个两位二进制数乘法器。

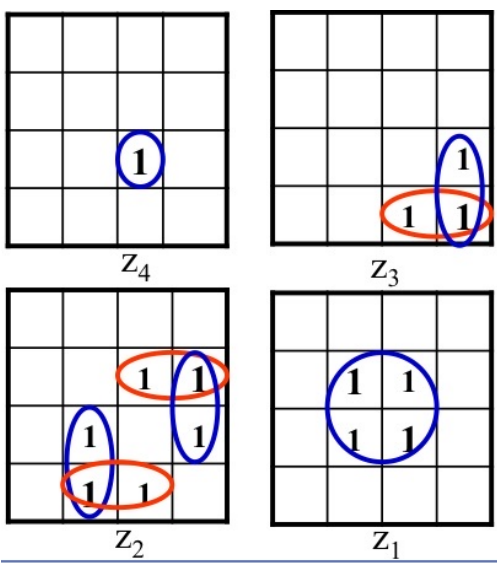
分析： 输入变量 $X = x_2x_1$ $Y = y_2y_1$

输出变量 $Z = z_4z_3z_2z_1$

一个卡诺图对应一个输出



x_2x_1	y_2y_1	$z_4z_3z_2z_1$
0 0	0 0	0 0 0 0
0 0	0 1	0 0 0 0
0 0	1 0	0 0 0 0
0 0	1 1	0 0 0 0
0 1	0 0	0 0 0 0
0 1	0 1	0 0 0 1
0 1	1 0	0 0 1 0
0 1	1 1	0 0 1 1
1 0	0 0	0 0 0 0
1 0	0 1	0 0 1 0
1 0	1 0	0 1 0 0
1 0	1 1	0 1 1 0
1 1	0 0	0 0 0 0
1 1	0 1	0 0 1 1
1 1	1 0	0 1 1 0
1 1	1 1	1 0 0 1



输出函数：

$z_4 = x_2x_1\ y_2y_1$

$z_3 = x_2\bar{x}_1y_2 + x_2y_2\bar{y}_1$

$z_2 = x_2\bar{x}_1y_1 + x_2\bar{y}_2y_1 + \bar{x}_2x_1y_2 + x_1y_2\bar{y}_1$

$z_1 = x_1y_1$

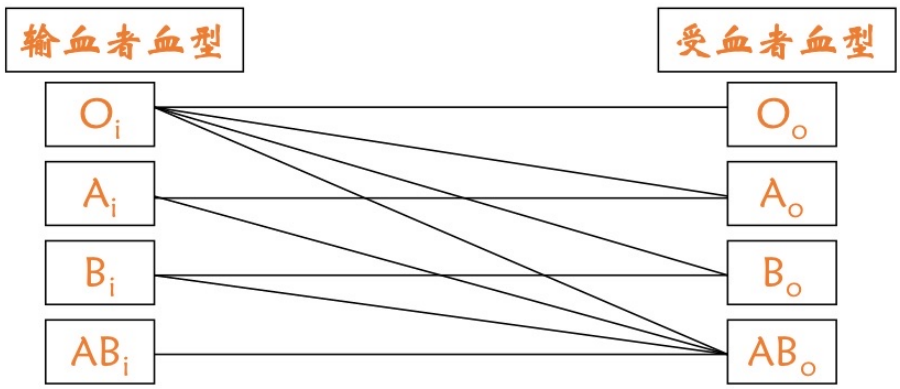
设计例2： 血液配型判别器。

分析：

输入变量： 输血者血型 A_i、B_i、AB_i、O_i

受血者血型 A_o、B_o、AB_o、O_o

输出变量： 配血成功 F



$F = O_i + A_i(A_o + AB_o) + B_i(B_o + AB_o) + AB_i\,AB_o$

设计思考题：开关控制电路

某一集体宿舍，共住有8人，共用一套照明设施，每人床头都有一个控制开关，要求每按动开关一下，就改变一次灯的状态(即亮→灭、灭→亮)。试设计出此灯控开关电路。

分析：输入变量K_0、K_1、K_2、K_3、K_4、K_5、K_6和K_7

输出变量 F

设开关初态F = 0，当$K_0K_1K_2K_3K_4K_5K_6K_7$均为状态 0

异或

设计思考题：码制转换电路

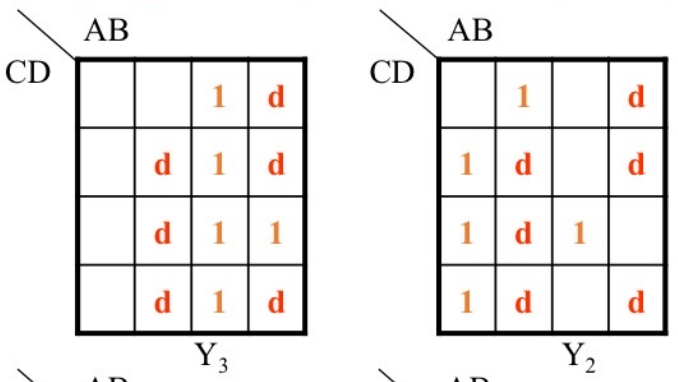
例1 输入为2421码，用“ABCD”指代；
输出为余3码用“$Y_3Y_2Y_1Y_0$”指代。

分析过程：

第一步，列出真值表

十进制数	ABCD	$Y_3Y_2Y_1Y_0$
0	0000	0011
1	0001	0100
2	0010	0101
3	0011	0110
4	0100	0111
	0101	d
	0110	d
	0111	d
	1000	d
	1001	d
	1010	d
5	1011	1000
6	1100	1001
7	1101	1010
8	1110	1011
9	1111	1100

第二步：按Y_3、Y_2、Y_1和Y_0分别填完卡诺图

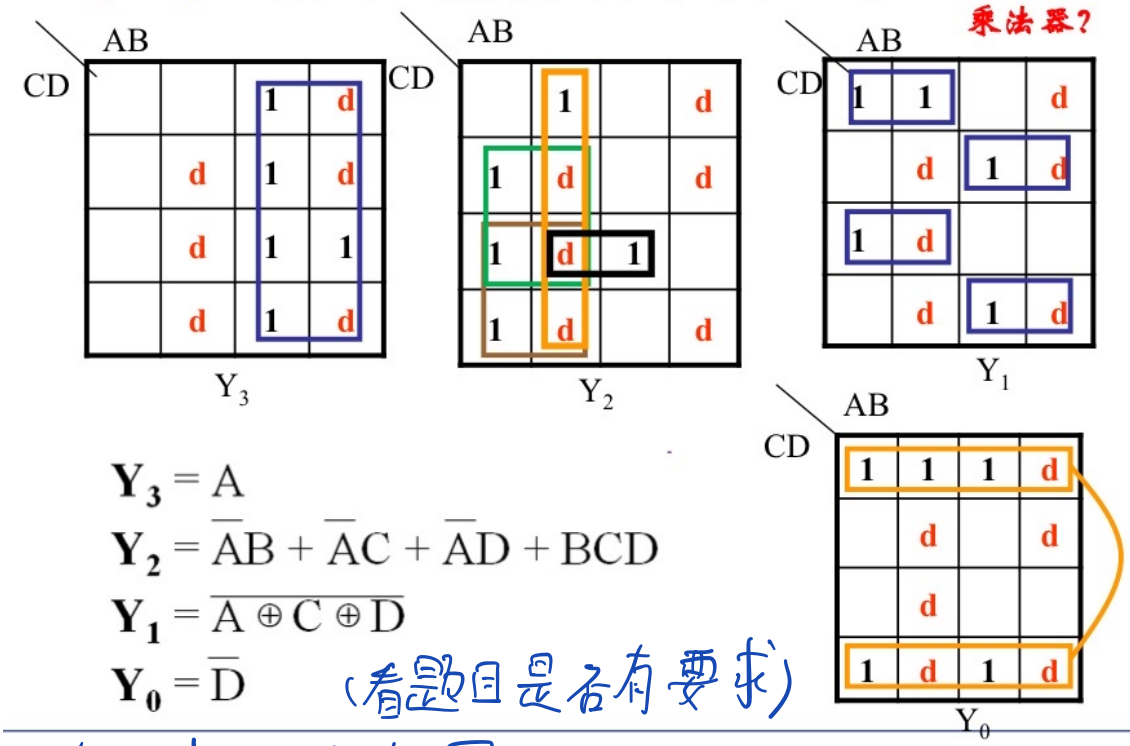


ABCD	$Y_3Y_2Y_1Y_0$
0000	0011
0001	0100
0010	0101
0011	0110
0100	0111
0101	d
0110	d
0111	d
1000	d
1001	d
1010	d
1011	1000
1100	1001
1101	1010
1110	1011
1111	1100

几个输出，几个卡诺图

第三步：找出最小覆盖并写出最简表达式

设计一个2位乘法器？



$$Y_3 = A$$

$$Y_2 = \overline{A}B + \overline{A}C + \overline{A}D + BCD$$

$$Y_1 = \overline{A \oplus C \oplus D}$$

$$Y_0 = \overline{D}$$

（看题目是否有要求）

第四步：画电路图

四、Verilog：

- **Verilog HDL程序基本结构**

```
module 模块名(端口名1，端口名2，端口名3，...);
    端口类型说明(input, output, inout);
    参数定义(可选);
    数据类型定义(wire, reg等);

    实例化低层模块和基本门级元件;

    连续赋值语句(assign);

    过程块结构(initial和always)
        行为描述语句;
endmodule
```

模块说明部分

结构级描述

数据流描述

行为级描述

逻辑功能描述部分

§2.3 组合电路中的竞争与险象

一、竞争：

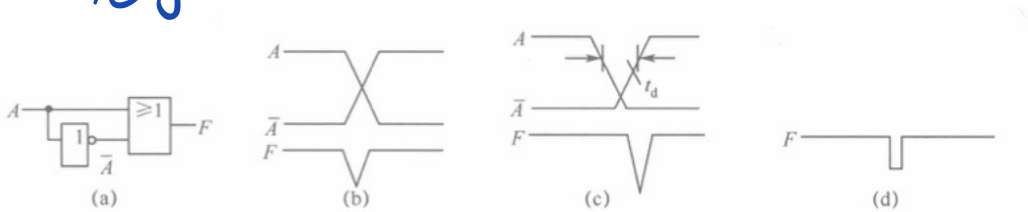
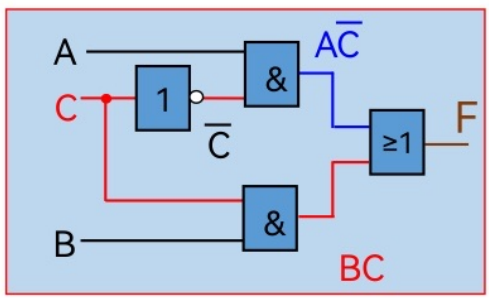


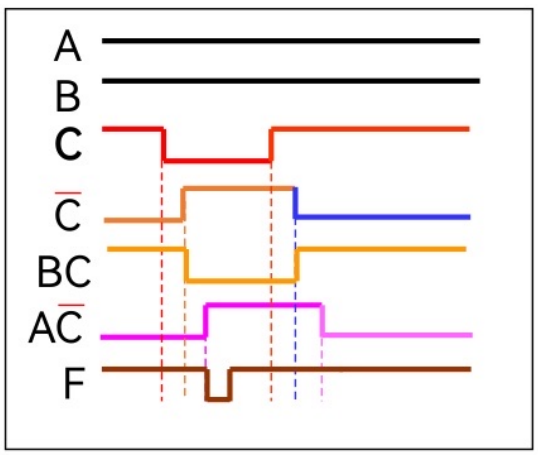
图 2.32　信号边沿及门的时延产生的尖峰脉冲

1. 竞争：同一信号或同时变化的某些信号，经过不同路径到达某一点有时差

2. 分类：错误输出：临界竞争
　　未错误输出：非临界竞争

例：　例　　$F = A\bar{C} + BC$ 且 $A = B = 1$ ，则逻辑上：$\bar{F}$= C+C=1





二、险象：

险象：临界竞争导致短暂输出错误（一定是竞争结果）
　　[竞争不一定有险象]

1. 静态险象：只有1个"0"/"1"错误窄脉冲

(1) 功能险象：(无法消除)

产生功能险象的条件是：

（1）$K(K>1)$个输入信号同时发生变化；

（2）K 个变化的变量的组合将使对应的输出既可能是“1”，又可能是“0”；

（3）输入变量变化前后的稳态输出相同。

例：

例 如图所示 $F = A\overline{C} + BC$，设 A=1

当 BC：00 →11 时，F应当恒为1

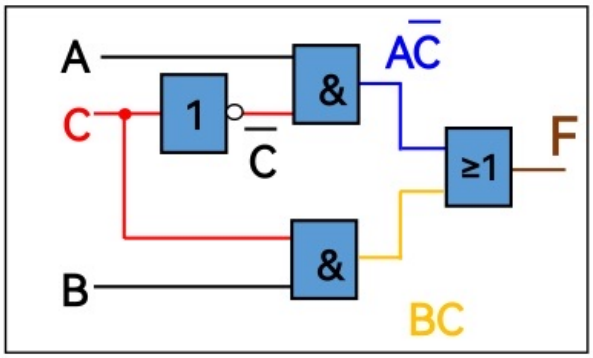
情况一：ABC：100 →110 →111

则 F：1 →1 →1 无险象

情况二：ABC：100 →101 →111

则 F：1 →0 →1 有险象

0	0	1 ←	1
0	1	1 ↓	0 ←



(2) 逻辑险象：

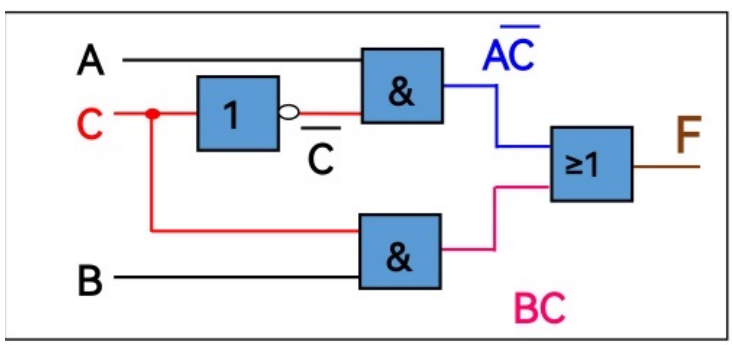
产生逻辑险象的条件是：

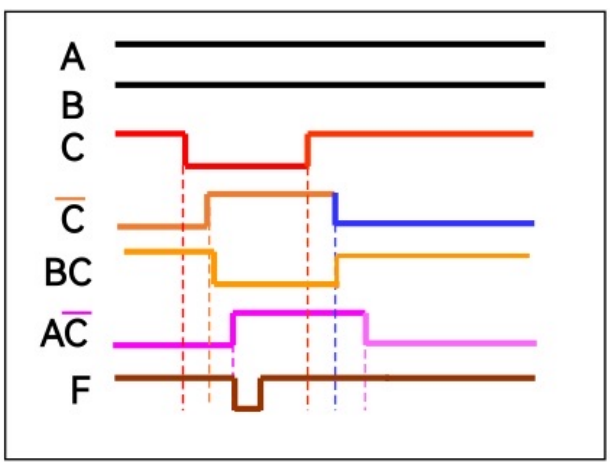
（1）仅有一个输入信号发生变化；

（2）输入变量变化前后的稳态输出相同；

例 如图 $F = A\overline{C}+BC$，当 A = B = 1，$F = \overline{C} + C \equiv 1$。实际上

当 C：1 → 0 时 F 产生险象。





总之，静态险象的产生是：

由于输入信号经过不同的路径又汇合到同一个门上的竞争所引起的。

在电路中，若输入信号变化前后，稳态的输出均为逻辑状态1（即高电平），且在状态“1”对应的高电平输出上出现一个负向窄脉冲，则该险象称为静态 1 险象，如前例所示 。

反之若稳态输出为逻辑状态0，且在状态“0”对应的低电平输出上出现一个正向窄脉冲，则该险象称为静态 0 险象，如下例所示。

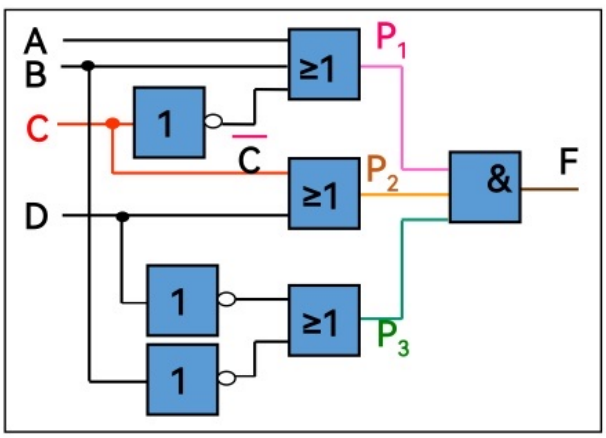
静态1险象：两级与或、两级与非-与非逻辑电路，1—>0
静态0险象：两级或与、两级或非-或非逻辑电路，0—>1

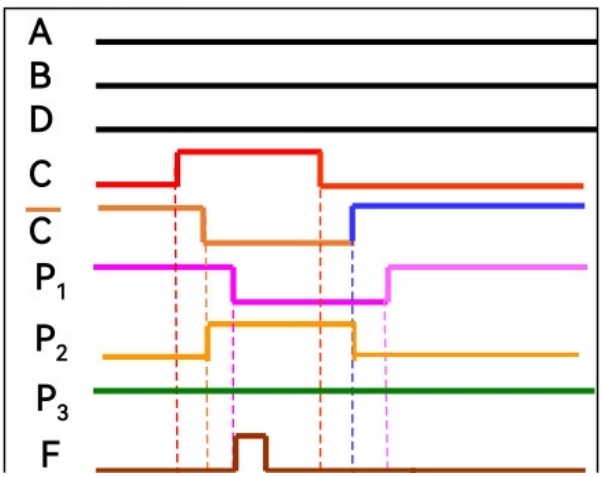
例 具有静态 0 险象的电路及时间图

如图所示 $F = (A + B + \overline{C})(C + D)(\overline{B} + \overline{D})$

当 $A = B = D = 0$ 时， 则 $F = C \cdot \overline{C} \equiv 0$

实际上，当 C： 0 → 1 时，F 产生静态 0 险象。





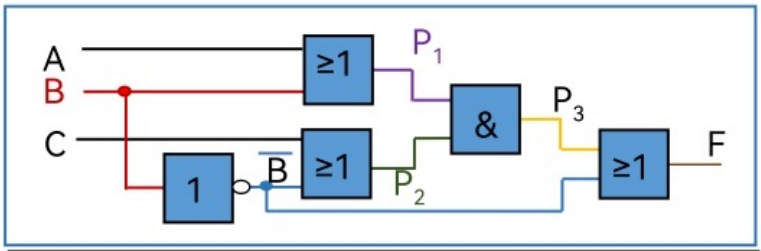
2. 动态险象：

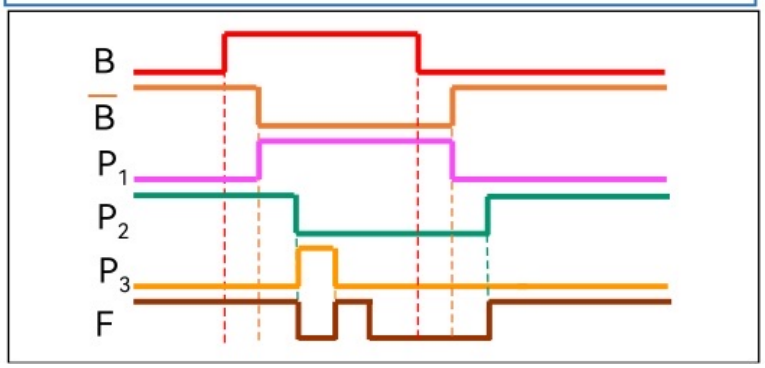
在多级组合逻辑电路中，若输入信号的变化通过多条路径向输出端汇合时，使在输出稳定之前输出变化三次,其间经过暂时状态0、1或者1、0，这种险象称之为动态险象。

输入变化的第一次会合只可能产生静态险象，只有在产生了静态险象，输入变化的再一次会合，才有可能产生动态险象。动态险象是由静态险象引起的，它也是竞争的结果。同样，在两级“与或”和“或与”电路中都不会发生。

消除了静态险象，则动态险象也不会出现!

例 $F = (A+B)(\overline{B}+C) + \overline{B}$ ，当A = C = 0 时， $F = B\overline{B} + \overline{B}$





F: 1—>0—>1—>0

三、险象的判别：

1. 卡诺图法：（判别两级与或/或与电路）

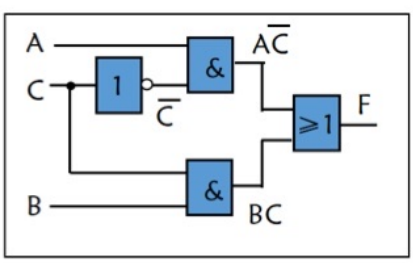
(1) 静态 1 险象判别

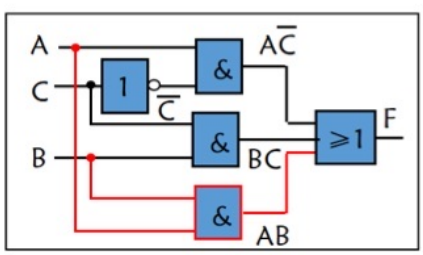
在两级“与或”电路或两级“与非-与非”电路中只可能出现静态 1 险象。(思考题)

在卡诺图中，与或式中的每个与项对应于圈1的一个卡诺圈，如果两个卡诺圈存在着部分“相切”，而这个“相切”的部分又没有被另外的卡诺圈所包含，则该电路必然存在静态 1 险象。

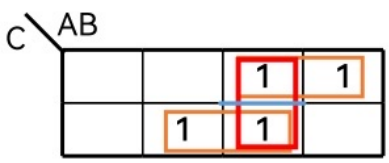
例 静态 1 险象的判别与消除电路

$F = A\overline{C} + BC$ 当 A = B = 1 时





与×→逻辑1
或×→逻辑0



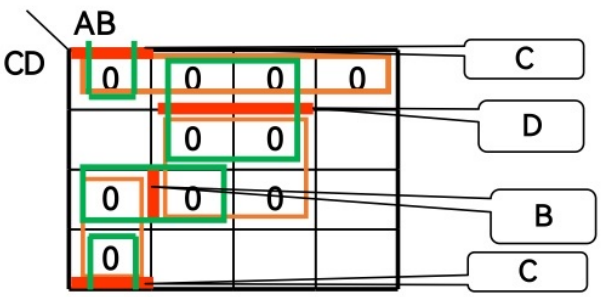
增加的与项AB，当AB=11时，无论 C 如何变化，使输出F总是1，从而消除了险象。

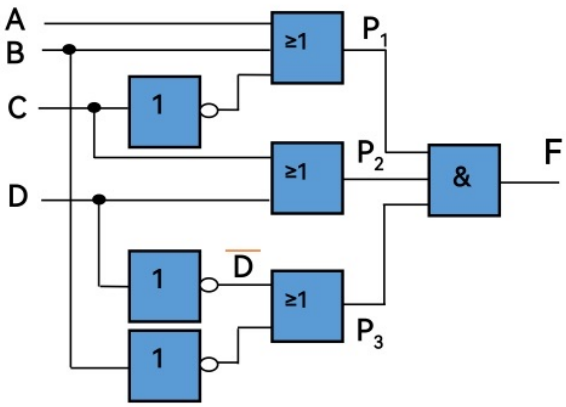
(2) 静态 0 险象判别

在两级“或与”电路或两级“或非—或非”电路中只可能出现静态 0 险象。（思考题）

在卡诺图中，按照圈0单元的卡诺圈是否存在着部分相切，而这个相切的部分又没有被另外的卡诺圈所包含，则该电路必然存在静态 0 险象。

例 如图所示电路 $F = (A+B+\overline{C})\ (C+D)\ (\overline{B}+\overline{D})$





两圈相切于在变量D的交替面

两圈相切于在变量B的交替面

两圈相切于在变量C的交替面

有三处相切，对应着三个静态0险象；分别增加三个卡诺圈，使输出F总是0，从而消除险象。

$$F = (A+B+\overline{C})\ (C+D)\ (\overline{B}+\overline{D})\ (A+B+D)\ (A+\overline{C}+\overline{D})\ (\overline{B}+C)$$

2. 逻辑表达式法：

如果电路中存在出现险象的可能性，其逻辑表达式有如下特点：

(1) 当某一变量同时以原变量和反变量的形式出现在逻辑表达式中，则该变量就具备了竞争的条件。

(2) 保留被研究变量，消去其他变量(其他变量取某些固定值，这些固定值是被研究变量产生竞争的条件)。

(3) 若得到的表达式为下列形式之一，则有险象存在：

$F = A + \overline{A}$ —— 静态 1 险象(如A从1→0)

$F = A \cdot \overline{A}$ —— 静态 0 险象(如A从0→1)

$$F = \left\{ \begin{array}{l} A \cdot (A + \overline{A}) \\ \overline{A} \cdot (A + \overline{A}) \\ A + A \cdot \overline{A} \\ \overline{A} + A \cdot \overline{A} \end{array} \right\} \text{—— 动态险象}$$

例1 $F = A\overline{C} + BC$ 式中变量 C 具备竞争条件

当 AB = 11时，C从 1→0，产生静态 1 险象

例2 $F = A\overline{C} + BC + AB$ 式中变量 C 具备竞争条件

例3 $F=(A+B+\overline{C})(C+D)(\overline{B}+\overline{D})$

① 变量B具备竞争条件：当ACD=011时

$F=B\cdot\overline{B}$ 存在静态0险象

② 变量C具备竞争条件：当ABD=000时

$F=\overline{C}\cdot C$ 存在静态0险象

③ 变量D具备竞争条件：当BC=10时

$F=D\cdot\overline{D}$ 存在静态0险象

例4 $F=(A+\overline{B})(B+C)+\overline{B}$

B如何变换，F有险象?

式中变量B具备竞争条件，当AC=00时，$F=\overline{B}\cdot B+\overline{B}$

四.险象的消除(逻辑险象)

1.增加多余项(与项)或乘多余因子(或项)

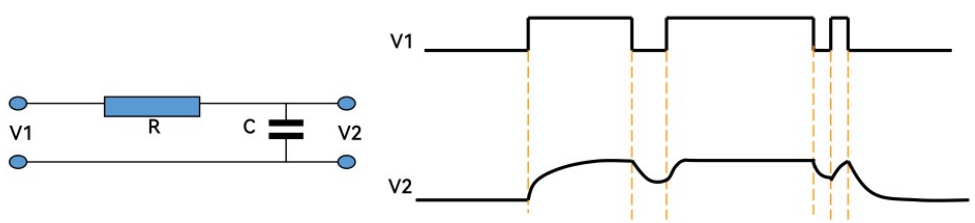
例 $F=A\overline{C}\overline{D}+\overline{B}C+BD$

将相切的部分均用多余的卡诺圈包含起来，则可消除静态1险象，得到：

$$F=A\overline{C}\overline{D}+\overline{B}C+BD+A\overline{B}\overline{D}+AB\overline{C}+CD$$

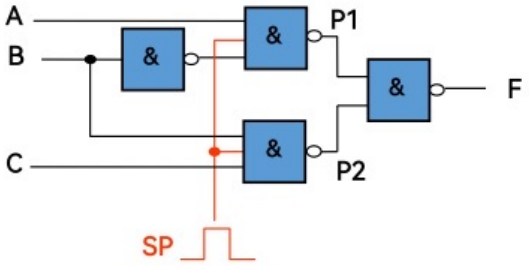
2.在输出端连接低通环节以减弱干扰

用低通滤波电路滤掉矮脉冲干扰，但将使输出变化的上升、下降沿增大，降低工作速度也使信号质量变坏。

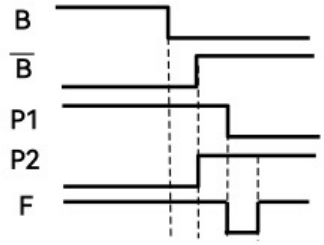


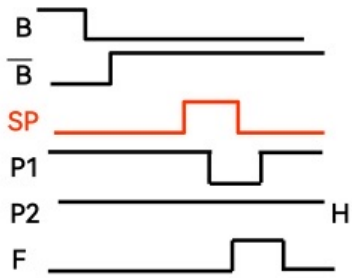
注意选择合适的R、C参数!

3. 利用取样脉冲避开险象：



改电平控制为脉冲控制。





在特定的电路系统中采用的方法。在一般的组合电路中慎用。选通信号的产生时机和脉冲宽度很重要，否则适得其反!

§2.4 常用的MSI组合逻辑器件

一、译码器和编码器：

1.译码器：

最常用的译码器是二进制译码器，又称为 n 线-2^n 线译码器，其输入代码为 n 位二进制数，输出代码为 2^n 取 1 码。也就是说，译码器输出为 2^n 个最小项。

例 2-4 译码器

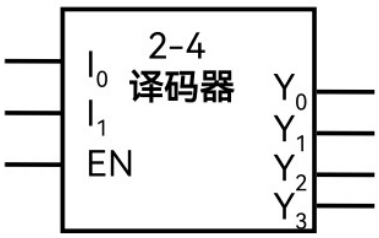
2-4译码器电路实现需要几种逻辑门?

输入代码字：I_1、I_0 ；**输入使能**：EN

输出代码字：Y_3 Y_2 Y_1 Y_0

注意信号编码和引脚的词典序!

功能描述：当 EN = 1，输入代码字是i的二进制表示，则输出Y_i (i为十进制数)位为1，其他位均为0的编码。



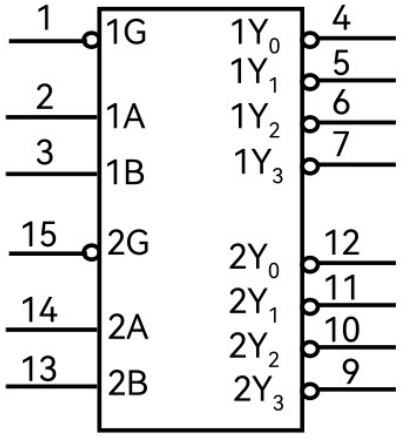
逻辑框图

真值表

输 入			输 出			
EN	I_1	I_0	Y_3	Y_2	Y_1	Y_0
0	d	d	0	0	0	0
1	0	0	0	0	0	1
1	0	1	0	0	1	0
1	1	0	0	1	0	0
1	1	1	1	0	0	0

(1) 双2-4译码器74LS139

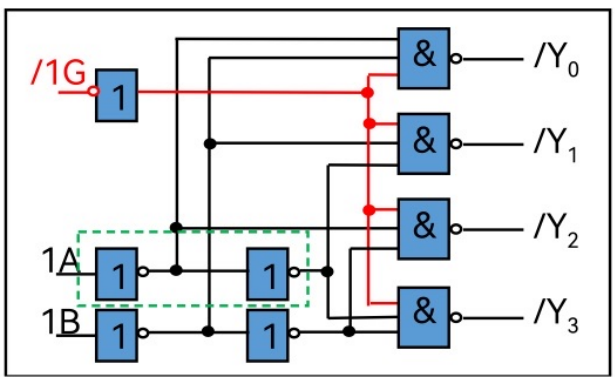
74LS139



引脚反字典序!

输 入			输 出			
/G	B	A	$/Y_3$	$/Y_2$	$/Y_1$	$/Y_0$
1	d	d	1	1	1	1
0	0	0	1	1	1	0
0	0	1	1	1	0	1
0	1	0	1	0	1	1
0	1	1	0	1	1	1

内部逻辑电路信号高有效!



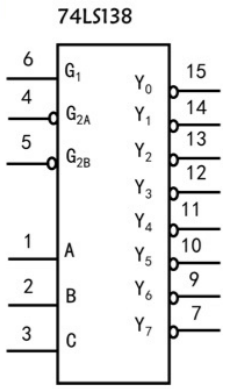
MSI?

增加信号驱动能力

(2) 3-8 译码器74LS138

74LS138真值表（功能表）

输入						输出							
G_1	$/G_{2A}$	$/G_{2B}$	C	B	A	$/Y_7$	$/Y_6$	$/Y_5$	$/Y_4$	$/Y_3$	$/Y_2$	$/Y_1$	$/Y_0$
0	d	d	d	d	d	1	1	1	1	1	1	1	1
d	1	d	d	d	d	1	1	1	1	1	1	1	1
d	d	1	d	d	d	1	1	1	1	1	1	1	1
1	0	0	0	0	0	1	1	1	1	1	1	1	0
1	0	0	0	0	1	1	1	1	1	1	1	0	1
1	0	0	0	1	0	1	1	1	1	1	0	1	1
1	0	0	0	1	1	1	1	1	1	0	1	1	1
1	0	0	1	0	0	1	1	1	0	1	1	1	1
1	0	0	1	0	1	1	1	0	1	1	1	1	1
1	0	0	1	1	0	1	0	1	1	1	1	1	1
1	0	0	1	1	1	0	1	1	1	1	1	1	1



译码器74LS138的使用要点

- 74LS138的输出信号为低有效，它有三个使能输入端（G_1、$/G_{2A}$、$/G_{2B}$），当使能信号均有效时，才产生有效输出。
- 74LS138的内部功能可用逻辑表达式描述如下：

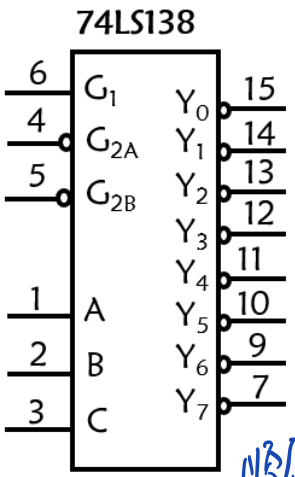
  $Y_i = G_1 \cdot G_{2A} \cdot G_{2B} \cdot m_i$

  其中，Y_i为内部输出编码字的第i位，

  m_i为输入变量C、B、A的最小项（请大家注意三变量的排列顺序“C B A”）。
- 74LS138 外部信号之间的关系为：

  $/Y_i = G_1 \cdot /G_{2A} \cdot /G_{2B} \cdot m_i$

如何用74LS138构造4-16译码器？

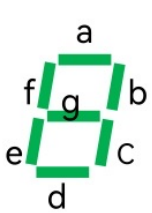


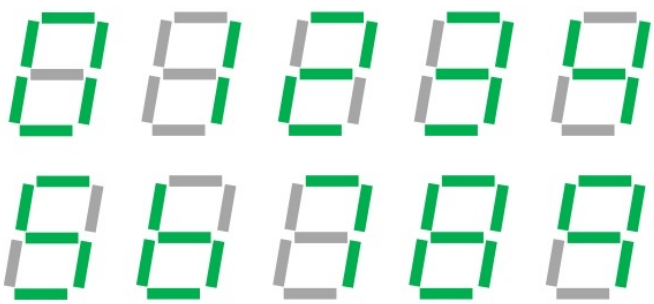
顺序：CBA → 110

A=0，B=1，C=1，哪位输出有效？Y_6

(3) BCD译码器74LS49

74LS49是常用的一种BCD码MSI器件，它的输入编码为4位BCD码，输出为7位编码字。它的输出代码不是仅有1位有效。





七段显示器件结构（有时也驱动八段显示）

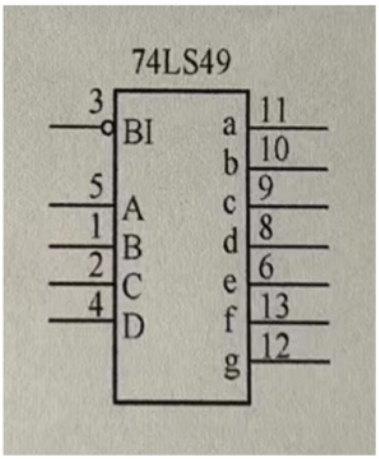
输入					输出							
	/BI	D	C	B	A	a	b	c	d	e	f	g
	0	d	d	d	d	0	0	0	0	0	0	0
0	1	0	0	0	0	1	1	1	1	1	1	0
1	1	0	0	0	1	0	1	1	0	0	0	0
2	1	0	0	1	0	1	1	0	1	1	0	1
3	1	0	0	1	1	1	1	1	1	0	0	1
4	1	0	1	0	0	0	1	1	0	0	1	1
5	1	0	1	0	1	1	0	1	1	0	1	1
6	1	0	1	1	0	0	0	1	1	1	1	1
7	1	0	1	1	1	1	1	1	0	0	0	0
8	1	1	0	0	0	1	1	1	1	1	1	1
9	1	1	0	0	1	1	1	1	0	0	1	1
	1	1	0	1	0	0	0	0	1	1	0	1
	1	1	0	1	1	0	0	1	1	0	0	1
	1	1	1	0	0	0	1	0	0	1	1	1
	1	1	1	0	1	1	0	0	1	1	1	1
	1	1	1	1	0	0	0	0	1	1	0	1
	1	1	1	1	1	0	0	0	0	0	0	0

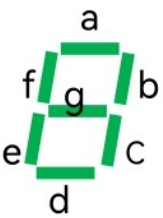
74LS49真值表

(BI：禁止显示控制端)

注意信号BI!

74LS49逻辑符号





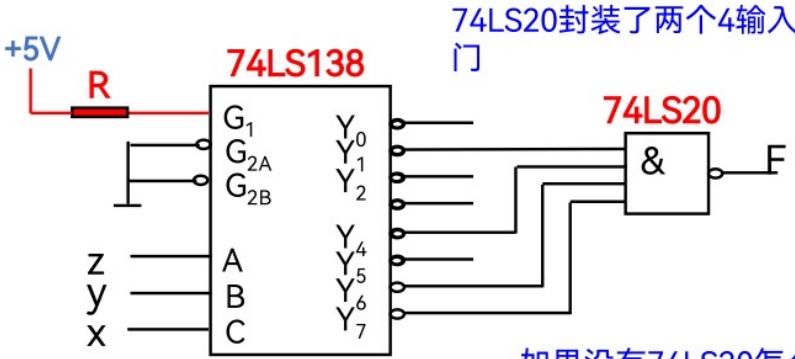
(4) 用MSI译码器实现组合逻辑函数

因为 n-2^n 二进制译码器的输出对应于 n 变量函数的2^n个最小项，
所以可以借用此器件来实现任何组合逻辑函数。（为什么？）

例1 用译码器74LS138实现 F(x,y,z) = Σm(1,4,6,7)

逻辑图如下所示：

74LS20封装了两个4输入的与非门



如果没有74LS20怎么办?

例2 用74LS138设计一个1位2进制全加器。

设：输入端分别为：被加数输入x_i、加数输入y_i、低位向本位的进位输入C_{i-1}

输出端分别为：本位的和输出S_i本位向高位的进位输出C_i

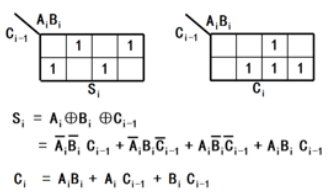
则一位全加器的真值表如下所示。

$C_{i-1}x_i\ y_i$	$S_i\ C_i$
0 0 0	0 0
0 0 1	1 0
0 1 0	1 0
0 1 1	0 1
1 0 0	1 0
1 0 1	0 1
1 1 0	0 1
1 1 1	1 1

由真值表得到：

$S_i = \sum m^3(1,2,4,7)$

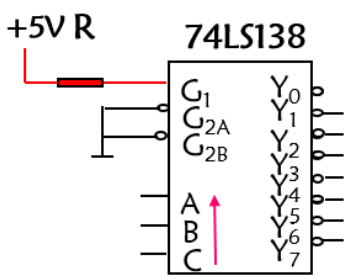
$C_i = \sum m^3(3,5,6,7)$



$S_i = A_i \oplus B_i \oplus C_{i-1}$

$= \overline{A}_i\overline{B}_i C_{i-1} + \overline{A}_i B_i \overline{C}_{i-1} + A_i\overline{B}_i\overline{C}_{i-1} + A_iB_i C_{i-1}$

$C_i = A_iB_i + A_i C_{i-1} + B_i C_{i-1}$



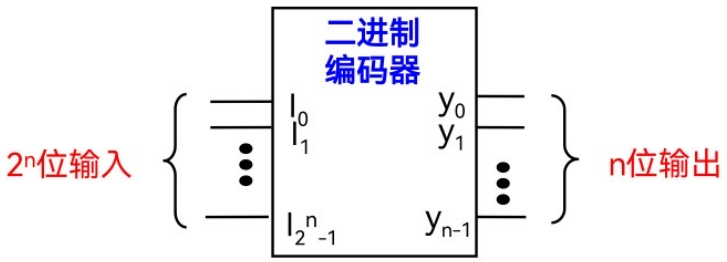
一位全加器逻辑图

2. 编码器：

当译码器的输出编码位数少于输入编码位数时，这种器件称为编码器。

2^n线-n线二进制编码器通用结构如图所示，其中输入端为2^n个，输出为 n 位二进制数，因此它的输入输出关系正好与译码器的相反。

约束条件：同一时刻只能有一个输入端有效（one hot码）。



二进制编码器框图

例如，一个8位输入、3位输出的编码器如图所示。

这是一个8-3二进制编码器。输入：I_0 ~ I_7，输出：Y_0 ~ Y_2

① 简化真值表

I_7 I_6 I_5 I_4 I_3 I_2 I_1 I_0	Y_2 Y_1 Y_0
0 0 0 0 0 0 0 1	0 0 0
0 0 0 0 0 0 1 0	0 0 1
0 0 0 0 0 1 0 0	0 1 0
0 0 0 0 1 0 0 0	0 1 1
0 0 0 1 0 0 0 0	1 0 0
0 0 1 0 0 0 0 0	1 0 1
0 1 0 0 0 0 0 0	1 1 0
1 0 0 0 0 0 0 0	1 1 1

② 输出函数表达式

$$Y_0 = I_1 + I_3 + I_5 + I_7$$
$$Y_1 = I_2 + I_3 + I_6 + I_7$$
$$Y_2 = I_4 + I_5 + I_6 + I_7$$

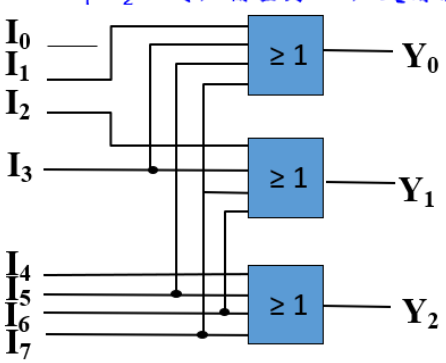
当且仅当输入代码中的一位为1，输出编码不可能重复。

I_1=I_2=1时，输出编码为多少？

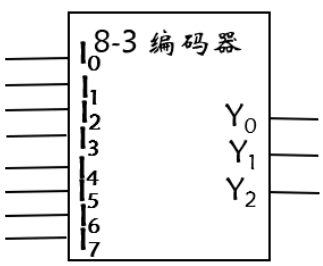
怎样保证？

③电路图 I_1=I_2=1时，输出为011，是错误的！

$Y_0 = I_1 + I_3 + I_5 + I_7$
$Y_1 = I_2 + I_3 + I_6 + I_7$
$Y_2 = I_4 + I_5 + I_6 + I_7$



④ 逻辑符号



I_i与Y_j之间的关系：使Y_j为 1 的是那些I_i呢？其下标 i的二进制数的第 j 位均为1。(From “0”)

例 $Y_1 = I_2+I_3+I_6+I_7$

即 $Y_1 = I_{010}+I_{011}+I_{110}+I_{111}$ Y_0、Y_2？

$$Y_0 = I_{001}+I_{011}+I_{101}+I_{111} = I_1+I_3+I_5+I_7$$
$$Y_2 = I_{100}+I_{101}+I_{110}+I_{111} = I_4+I_5+I_6+I_7$$

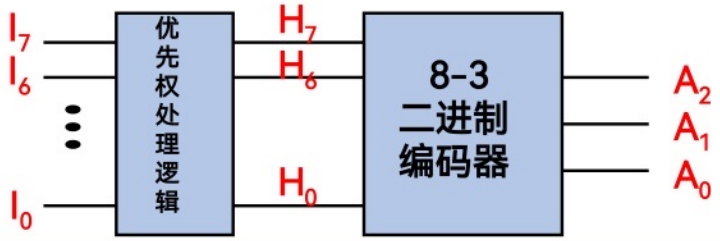
(1) 优先权编码器 *Priority Encoders*

如果在任一时刻，允许 2^n 个输入部件中有多个部件同时提出请求，则上页的 2^n–n 二进制编码器产生的 n 位编码必定有重复，而不能与输入请求的条件一一对应了。为此，应对输入端进行优先权分配，使编码器仅响应请求中优先权最高的有效输入端，并产生相应的输出编码。这种具有指定输入端优先权顺序的编码器。称为优先权编码器。

8-3 优先权编码器的结构框图如下所示：

如何设计优先权处理逻辑生成中间信号H_7~H_0?

设优先权为：I_7(最高)→ I_6→ I_5→ I_4→ I_3→ I_2→ I_1→ I_0



$$A_2 = H_4 + H_5 + H_6 + H_7$$
$$A_1 = H_2 + H_3 + H_6 + H_7$$
$$A_0 = H_1 + H_3 + H_5 + H_7$$

③ 优先权处理逻辑

设 优先权为：

I_7(最高) →I_6→ I_5→ I_4→ I_3→ I_2→ I_1→ I_0

$$H_7 = I_7$$
$$H_6 = \overline{I}_7 I_6$$
$$H_5 = \overline{I}_7 \overline{I}_6 I_5$$
$$H_4 = \overline{I}_7 \overline{I}_6 \overline{I}_5 I_4$$
$$H_3 = \overline{I}_7 \overline{I}_6 \overline{I}_5 \overline{I}_4 I_3$$
$$H_2 = \overline{I}_7 \overline{I}_6 \overline{I}_5 \overline{I}_4 \overline{I}_3 I_2$$
$$H_1 = \overline{I}_7 \overline{I}_6 \overline{I}_5 \overline{I}_4 \overline{I}_3 \overline{I}_2 I_1$$
$$H_0 = \overline{I}_7 \overline{I}_6 \overline{I}_5 \overline{I}_4 \overline{I}_3 \overline{I}_2 \overline{I}_1 I_0$$

④ 输出编码为：

$$/A_2 = \overline{H_4 + H_5 + H_6 + H_7}$$
$$/A_1 = \overline{H_2 + H_3 + H_6 + H_7}$$
$$/A_0 = \overline{H_1 + H_3 + H_5 + H_7}$$

输出使能为：

$$EO = \overline{I_0 + I_1 + \ldots + I_7}$$

$$/EO = \overline{/I_0 \,/ I_1 \ldots /I_7}$$

(2) MSI优先权编码器 74LS148

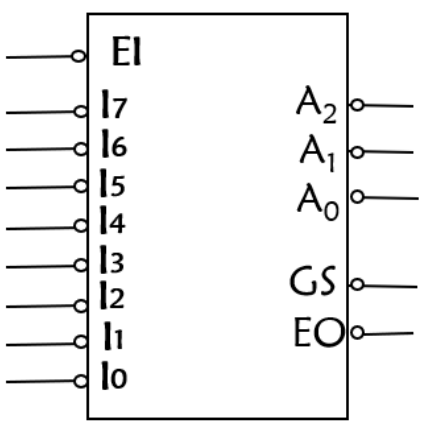
注意内部信号和外部输出信号的等效关系！

② 真值表

输 入									输 出				
/EI	/I_0	/I_1	/I_2	/I_3	/I_4	/I_5	/I_6	/I_7	/A_2	/A_1	/A_0	/GS	/EO
1	d	d	d	d	d	d	d	d	1	1	1	1	1
0	d	d	d	d	d	d	d	0	0	0	0	0	1
0	d	d	d	d	d	d	0	1	0	0	1	0	1
0	d	d	d	d	d	0	1	1	0	1	0	0	1
0	d	d	d	d	0	1	1	1	0	1	1	0	1
0	d	d	d	0	1	1	1	1	1	0	0	0	1
0	d	d	0	1	1	1	1	1	1	0	1	0	1
0	d	0	1	1	1	1	1	1	1	1	0	0	1
0	0	1	1	1	1	1	1	1	1	1	1	0	1
0	1	1	1	1	1	1	1	1	1	1	1	1	0

① 逻辑符号

74LS148



EO：选通输出端（输出使能端）

GS：优先扩展输出端（组选通输出端）

二、三态缓冲器：

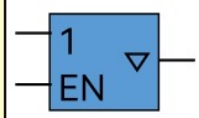
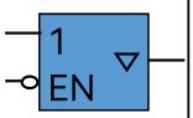
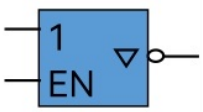
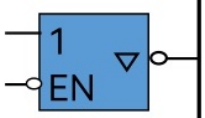
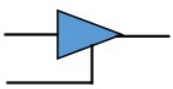
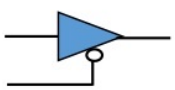
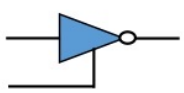
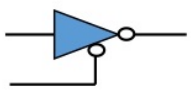
三态是指器件的输出有三种状态：即逻辑0 (L电平)、逻辑1 (H电平)和高阻抗状态(或悬浮态)。最基本的三态器件是三态缓冲器，又称为三态门或三态驱动器。

三角符号表示三态输出，它受使能输入（EN）端控制。

三态缓冲器可使多个数据源分时共享一根（条）公用线路：

为了避免多个数据源同时驱动共享线，则不能在使能一个数据源的同时使能另一个数据源。

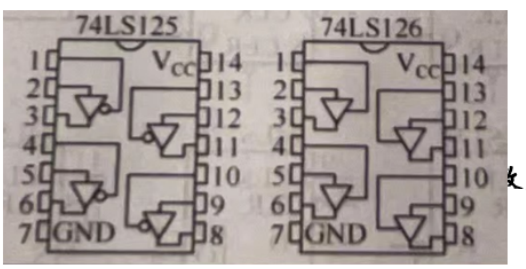
三态器件进入高阻态的速度比退出高阻态的速度快，保证器件断开和连通共享线有足够安全的无信号时间。

矩形符号				
变形符号				
	原码输出 高有效使能	原码输出 低有效使能	反码输出 高有效使能	反码输出 低有效使能

(1) 标准的SSI三态缓冲器

SSI：74LS125 使能端低有效。

74LS126 使能端高有效。



(见第1行)。

地址总线→单向

数据总线→双向

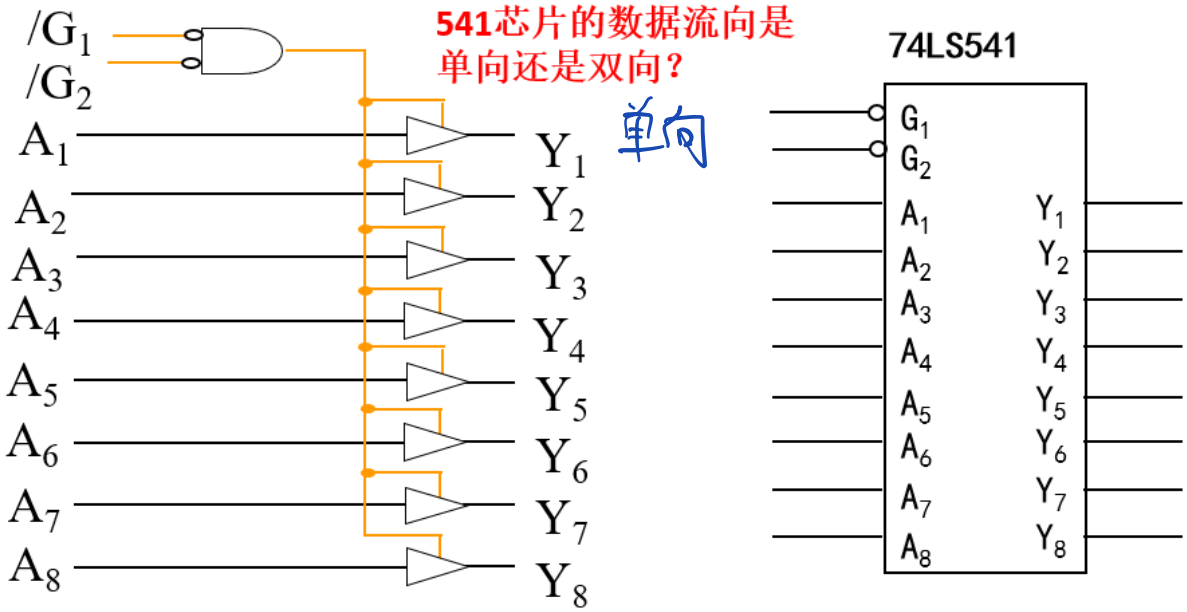
最常使用共享总线的场合是多位数据总线。

例如：在8位微处理机系统中，数据总线的宽度是8，并且外围器件通常一次放置（传输、传送）8位数据到总线上。这样外围器件都在同一时刻使能8个三态缓冲器，因此每个三态缓冲器独立的对外使能输入端都多余了。

为减少总线应用中三态缓冲器的芯片数及连线，三态缓冲器中包含多个三态缓冲器时常常共用使能输入。

(2) MSI 三态缓冲器

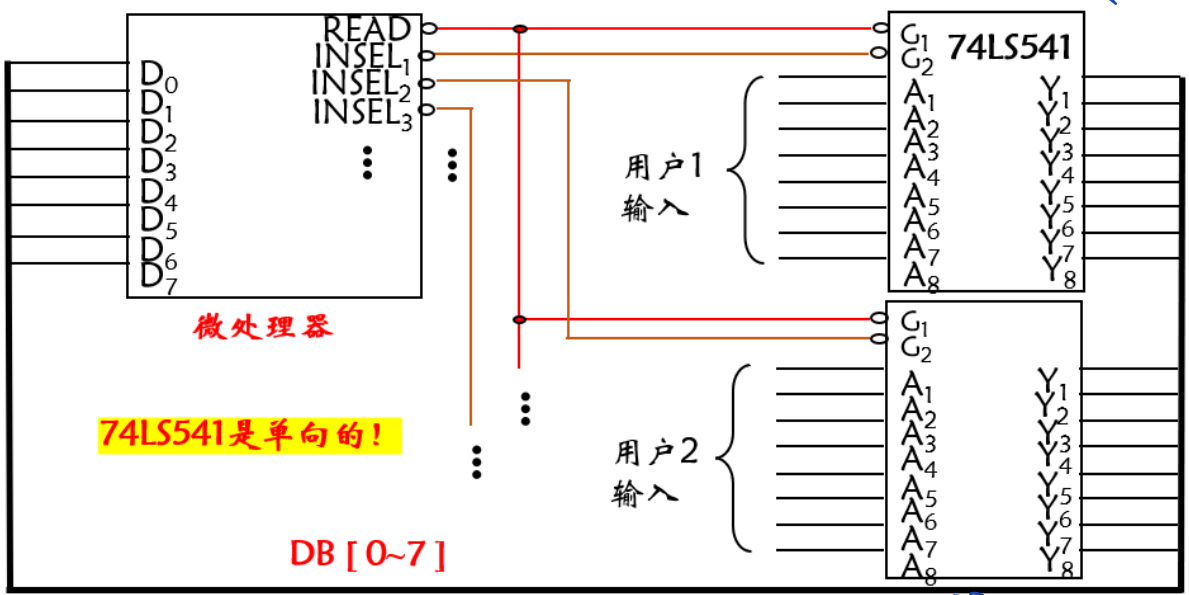
① 多端口输入，MSI 74LS541为八三态缓冲器



例如：MSI 74LS541（八三态缓冲器）的应用

MPU对用户设备写数据怎么办？

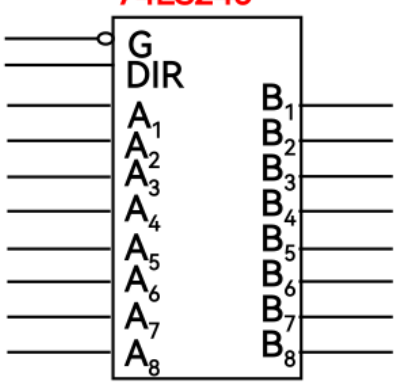
将输入数据导入微处理器



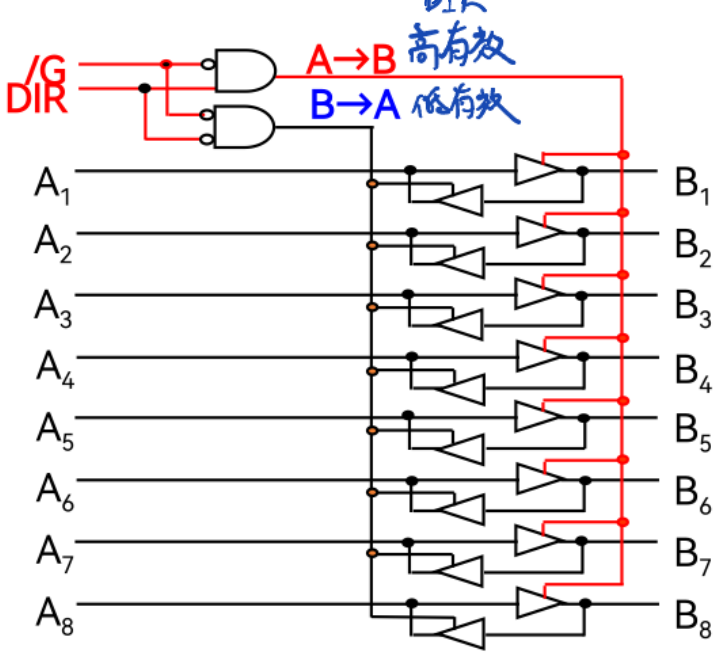
② 双向总线收发器

74LS245八三态
总线收发器

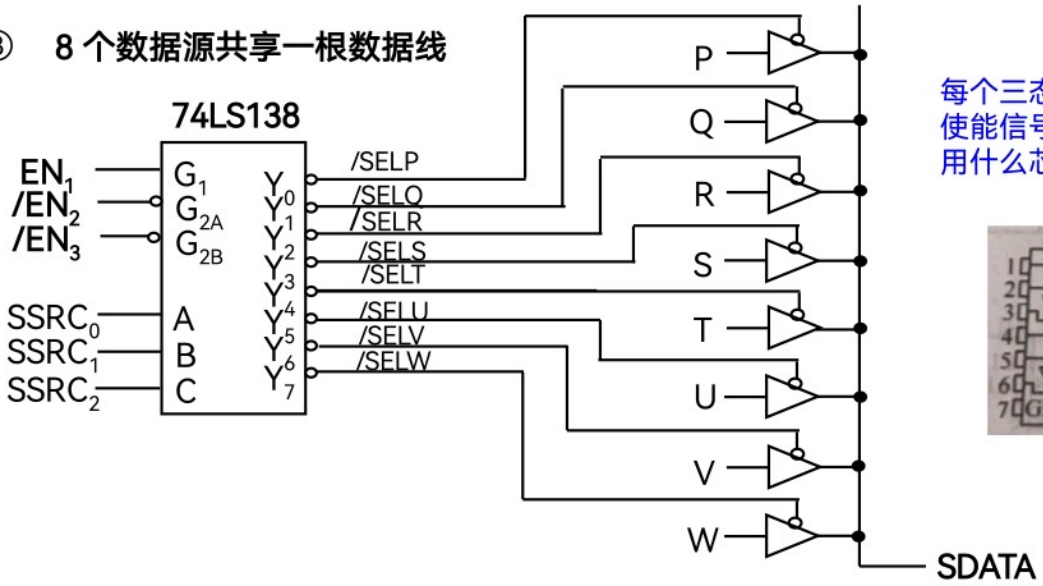
74LS245



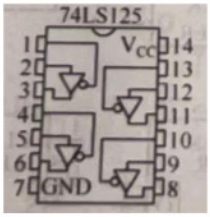
见书P81图2.65



③ 8 个数据源共享一根数据线

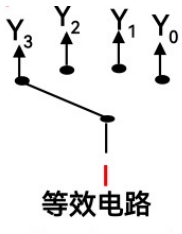


每个三态门是单独的使能信号控制，可以用什么芯片？



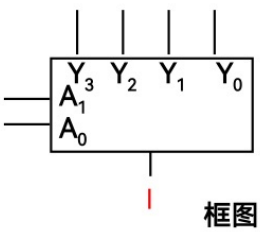
三、数据分配器和多路选择器：

1.数据分配器

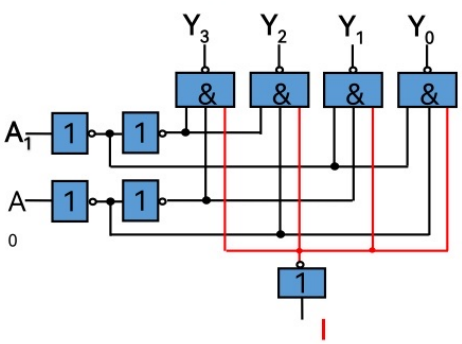


等效电路

（单刀多掷开关）



框图



逻辑电路图

图中 I：传送数据输入端；

A_1,A_0：地址码输入端；

y_3,y_2,y_1,y_0：输出的数据通道。

这种分配器被称为“1~4多路分配器”。

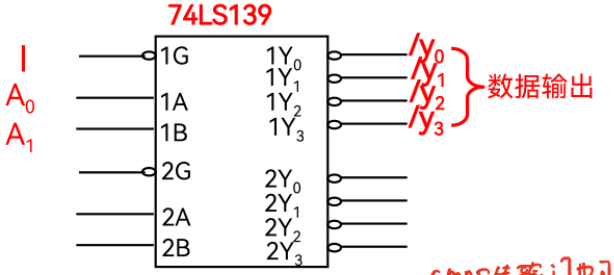
一般表达式为：$y_i = I$ 其中 i为地址码A_{n-1}……A_0对应的十进制值。

➤ 用二进制译码器作为数据分配器

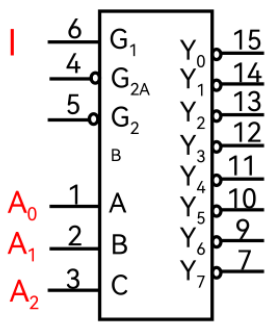
例 用1/2 74LS139作为四输出数据分配器。

- 将使能端G 作为数据输入端，即 I 接至G端；
- 将数据输入端作为地址输入端，即A端接地址A_0位；B端接地址A_1位。 $/Y_i = \overline{I}$

则： $/y_i = I$ i为地址码A_1A_0对应的十进制数。



CMOS传输门也可实现

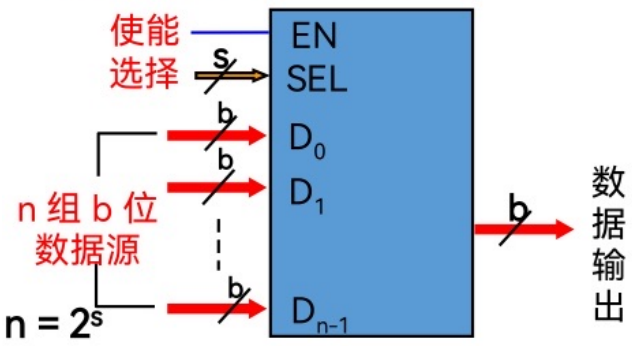


2. 多路选择器：

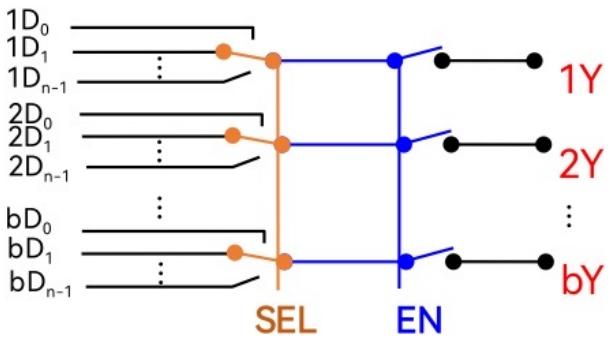
多路选择器又称数据选择器(MUX)，也是一个数字开关，可以从 n 路源数据中选择一路送至输出端。

假设有 n 组（路）输入数据源，每组（路）数据源的宽度为b 位二进制数，则反映多路选择器输出关系的框图和等效的电路如下图所示。其中高有效使能端 EN 的功能为：当EN = 0 时，所有的输出为 0。

① 多路选择器的结构框图



② 多路选择器的等效功能



③ 多路选择器输出逻辑表达式

从n组数据源中选择哪一组数据源传送到输出端，由选择输入端的输入值 S 决定。

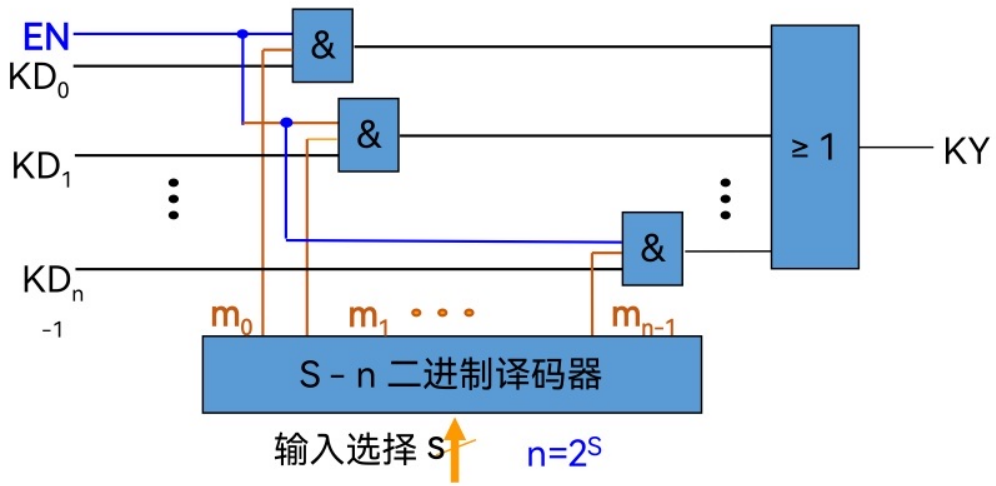
S与 n 的关系为：$n = 2^s$ （或 $S = \log_2 n$）

S 位选择信号有 2^s 种组合(即最小项)。每一种组合对应选择n ($= 2^s$)组输入数据源中的一组。 逻辑表达式为：

$$KY = \sum_{i=0}^{n-1} EN \cdot m_i \cdot KD_i \qquad K = 1,\ 2,\ \ldots,\ b$$

式中：KY为输出位，KD_i是第 i 组输入数据源的第K位，m_i 是 S 位选择输入变量的最小项。

④ 多路选择器的原理图（某位输出多路选择器,K=1~b）



(1) MSI 多路选择器

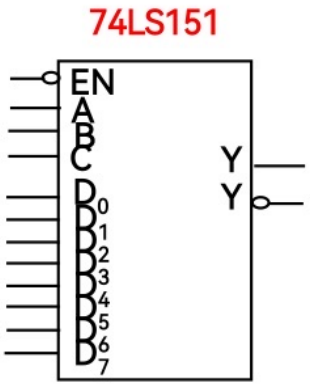
➤八输入 1 位输出多路选择器74LS151

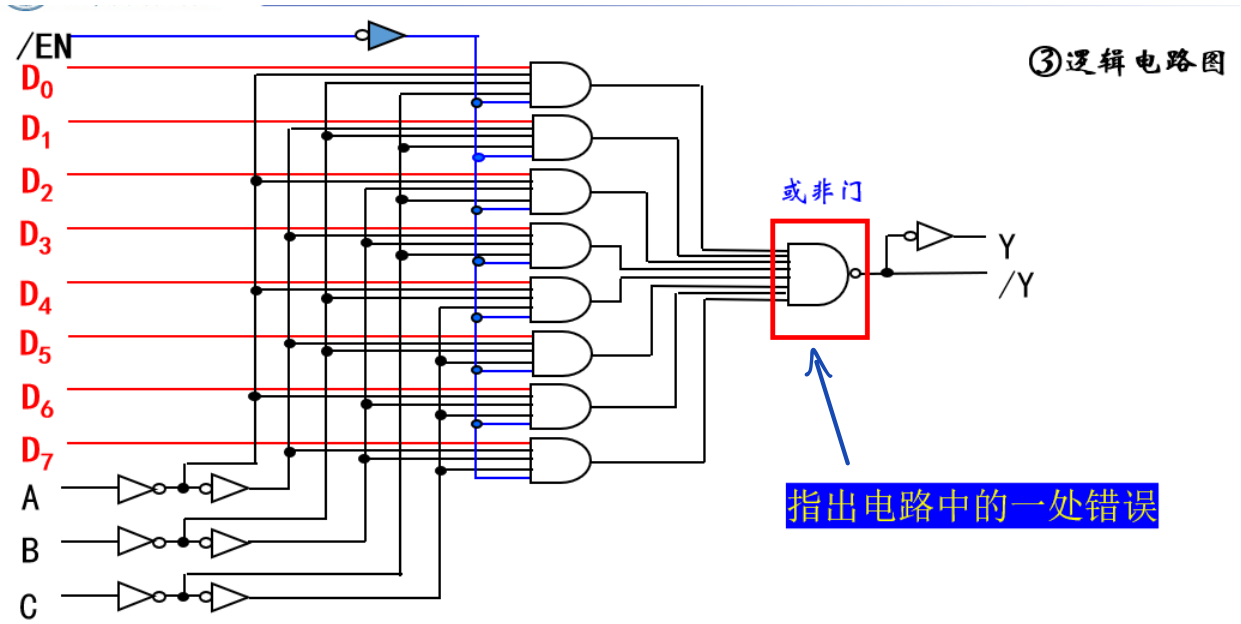
- 一个使能输入端/EN
- 三个选择输入端C、B、A
- 8 个数据输入端 $D_7 \sim D_0$
- 2 个互反输出 Y、/Y

① 简化真值表

输 入					输 出	
/EN	C	B	A	D	Y	/Y
1	d	d	d		0	1
0	0	0	0		D_0	$\overline{D_0}$
0	0	0	1		D_1	$\overline{D_1}$
0	0	1	0		D_2	$\overline{D_2}$
0	0	1	1		D_3	$\overline{D_3}$
0	1	0	0		D_4	$\overline{D_4}$
0	1	0	1		D_5	$\overline{D_5}$
0	1	1	0		D_6	$\overline{D_6}$
0	1	1	1		D_7	$\overline{D_7}$

② 逻辑符号



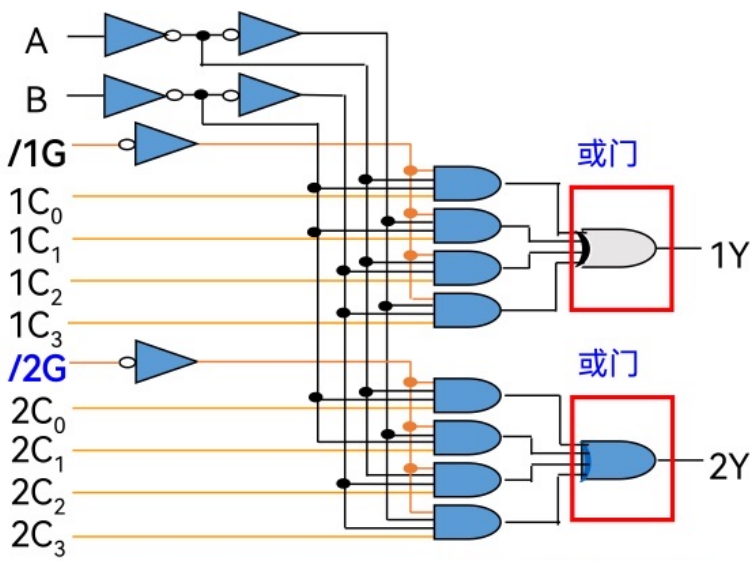


➤ 四输入 2 位多路选择器 74LS153

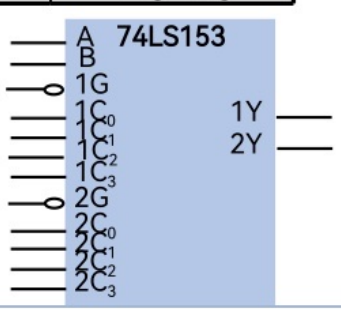
① 简化真值表

输 入				输 出	
/1G	/2G	B	A	1Y	2Y
1		d	d	0	0
0		0	0	$1C_0$	$2C_0$
0		0	1	$1C_1$	$2C_1$
0		1	0	$1C_2$	$2C_2$
0		1	1	$1C_3$	$2C_3$

② 逻辑电路图



③ 逻辑符号



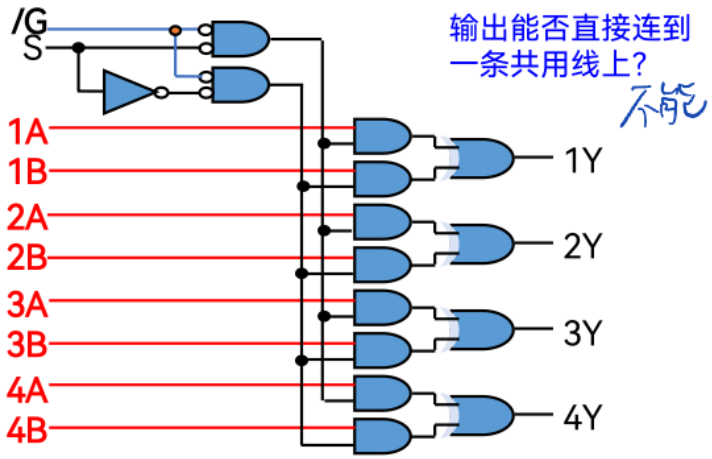
输出是单独控制的!

➤ 二输入4 位多路选择器 74LS157

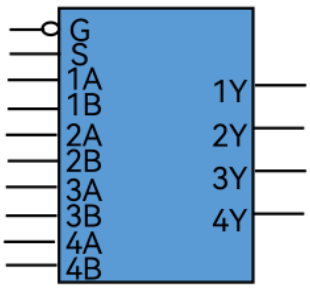
① 简化真值表

输 入		输 出			
/G	S	1Y	2Y	3Y	4Y
1	d	0	0	0	0
0	0	1A	2A	3A	4A
0	1	1B	2B	3B	4B

② 逻辑电路图



③ 逻辑符号



➤ 三态输出多路选择器

具有三态输出的多路选择器，当其使能输入无效时，将强制输出端处于高阻抗。

有三态输出端的多路选择器的输出端可以直接连接在一起，使得用这种器件可以方便地组成更大的多路选择器 MUX，常用的这种器件有74LS251，74LS253和74LS257等。

74LS151: INPUT:8 ,OUTPUT:1

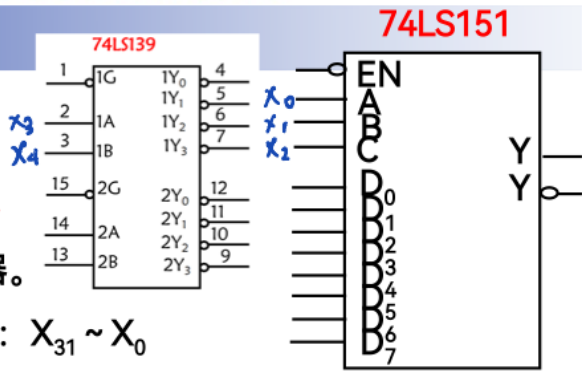
74LS153: INPUT:4 ,OUTPUT:2

74LS157: INPUT:2 ,OUTPUT:4

74LS251,74LS253, 74LS257: 为三态输出多路选择器。

西安交通大学 XI'AN JIAOTONG UNIVERSITY

(2) 多路选择器的扩展

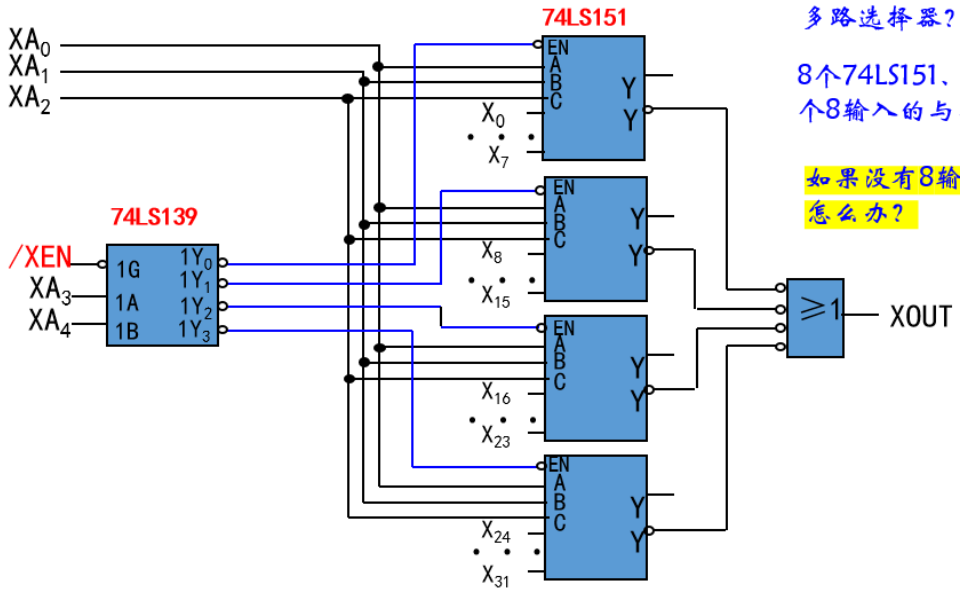


➤使用无三态输出的多路选择器及译码器

例：设计一个32位输入 1 位多路选择器。

- 5个选择输入：$XA_4 \sim XA_0$　32路输入：$X_{31} \sim X_0$
- 采用 4 个74LS151，每个器件可处理8个输入，这样将输入分为4组，每组由一个74LS151处理
- 选择输入的低三位$XA_2 \sim XA_0$连接到 4 个74LS151的C、B、A端，决定组内选择
- 选择输入的高二位XA_4、XA_3通过一级2-4译码器1/2 74LS139产生 4 个输出，每个输出连接到一个74LS151的使能输入端。

用74LS151组成的32输入1位多路选择器



如何用74LS151实现64输入1位多路选择器？

8个74LS151、1个74LS138和1个8输入的与非门

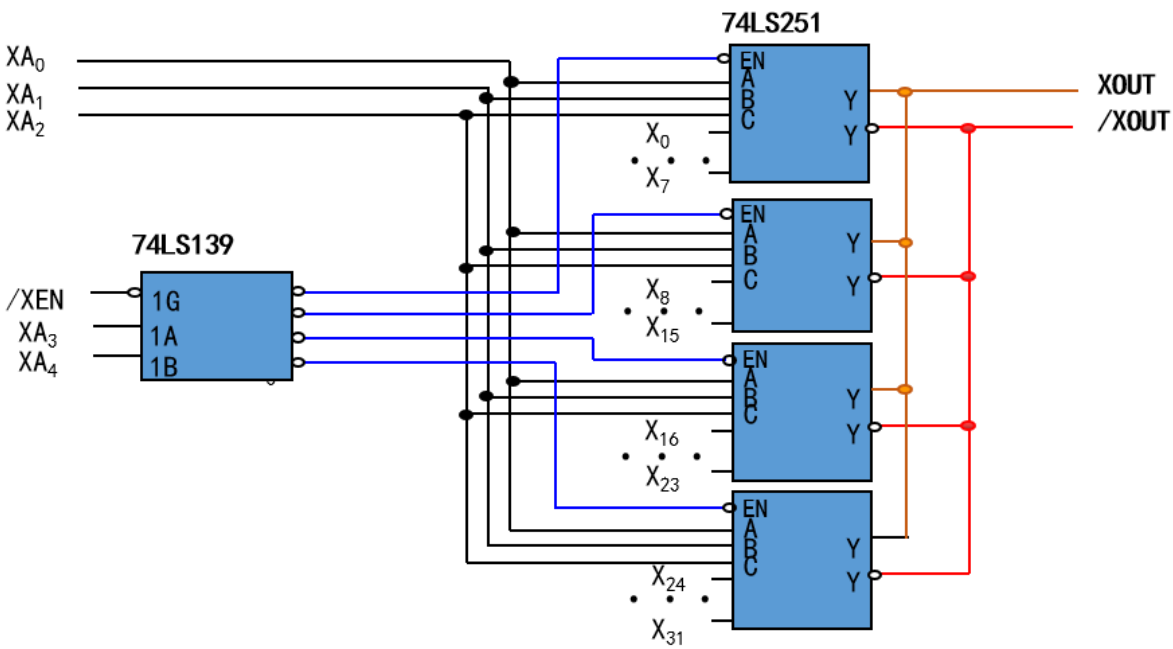
如果没有8输入与非门/或门怎么办？

➤ 使用三态输出的多路选择器及译码器

例：用74LS251设计一个32输入 1 位多路选择器。

- 当输出处于高阻态时，该输出线可以与其他输出线直接连接在一起，并且不影响其他输出线的高、低电平。
- 在任意时刻只能有一个74LS251被74LS139使能，此时输出线XOUT和/XOUT上的逻辑值就是该被使能的74LS251的输出值
- 当输入使能/XEN无效时，所有74LS251的输出为高阻态，输出线XOUT和/XOUT上的逻辑值不确定。

用74LS251组成的32输入1位多路选择器。



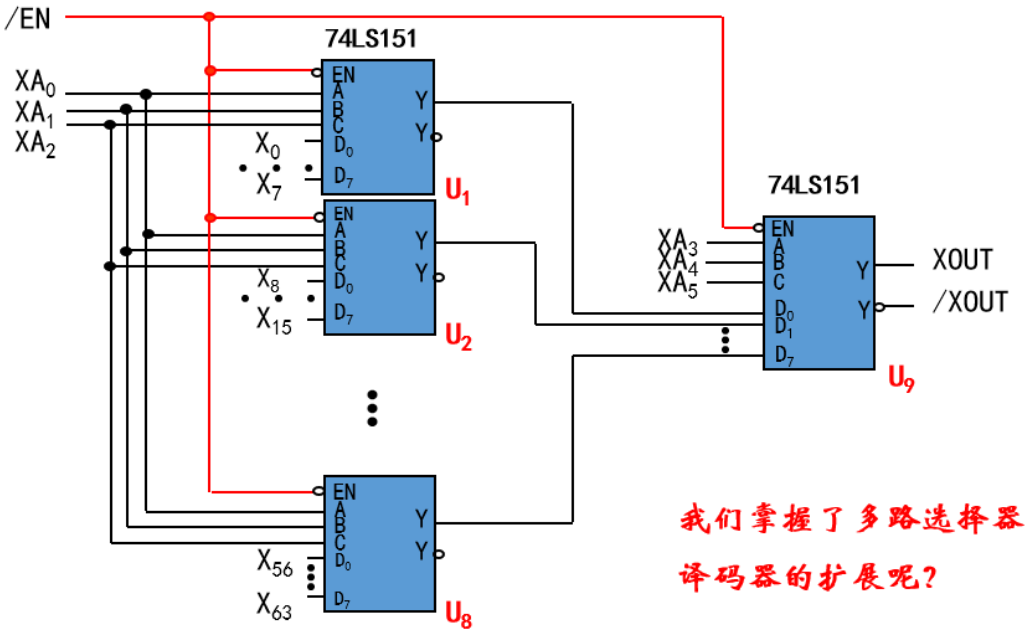
➤ 采用多级MUX的树形结构

- 将多路选择器MUX分级连接，低一级(前一级) MUX的输出作为其高一级(后一级) MUX的数据输入；
- 用选择输入信号的低位控制低一级MUX，高位控制高一级MUX
- 各级的使能输入用同一个信号进行控制。

译码器的输出信号是独立控制每一个MUX的使能端。

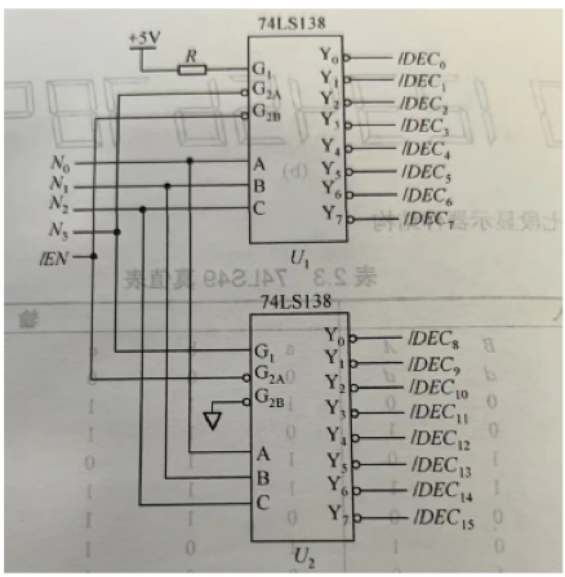
例 采用多级树形结构组成64输入 1 位多路选择器。

64输入1位多路选择器。



我们掌握了多路选择器的扩展使用，译码器的扩展呢？

用74LS138设计4-16译码器。



高位控制使能

N_3=0时，U2输出无效，U1输出m_0~m_7；
N_3=1时，U1输出无效，U2输出m_8~m_{15}

如何设计5-32译码器呢？

需要4个74LS138和1个1/2 74LS139（2-4译码器）；5位输入信号的高2位送到139芯片的输入，139芯片的输出作为4个138芯片的使能输入，低3位送到4个138芯片的输入端。

(3) 用多路选择器实现任意组合逻辑函数!

例1　$F(x,y,z) = \Sigma m^3(1,2,6,7)$

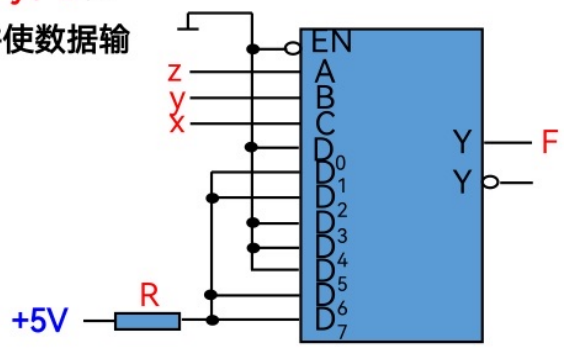
① 选择 S = 3 的MUX 74LS151，把 x、y、z 分别连到 74LS151 的 C、B、A 选择端，并使数据输入端为:

$D_0 = D_3 = D_4 = D_5 = 0$

$D_1 = D_2 = D_6 = D_7 = 1$

则输出端Y的输出即为F。

74LS138实现F还需要加一个与非门!



大家不要忘了芯片的使能信号

②用“四选1”多路选择器实现该三变量逻辑函数

将函数 F 改写成变量表达式：$F(x,y,z) = \sum m^3(1,2,6,7)$

$$=\bar{x}\,\bar{y}\,z+\bar{x}y\,\bar{z}+xy\,\bar{z}+x\,y\,z$$
$$=\bar{x}\,\bar{y}\,z+\bar{x}y\,\bar{z}+x\,y \quad \text{构造项，使x,y能够成为信号源}$$
$$=(\bar{x}\,\bar{y})\cdot z+(\bar{x}\,y)\cdot\bar{z}+(x\,\bar{y})\cdot 0+(x\,y)\cdot 1$$

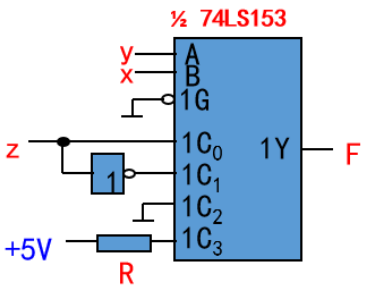
x、y作为地址选择变量，z、$\bar{z}$、0、1作为MUX的源数据输入D，则有：

$$F=\sum_{i=0}^{3} m_i D_i$$
$$= m_0 D_0 + m_1 D_1 + m_2 D_2 + m_3 D_3$$

式中 m_i 为x、y的最小项，D_i为：

$D_0 = z$，$D_1 = \bar{z}$，$D_2 = 0$，$D_3 = 1$

电路逻辑图如图所示（图中对应的输入为：$1C_i$）。



例2　$F(x,y,z) = \Sigma m^3(1,2,3,6)$

选用 4 输入 1 位多路选择器74LS153。

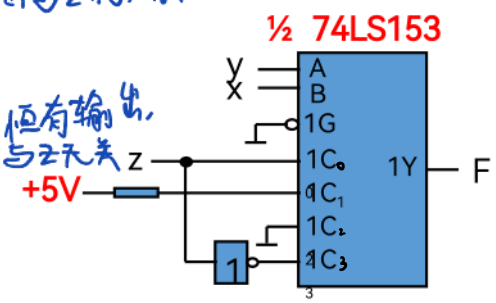
① 列出函数F的真值表（仅取值为1的部分）

输出与z的关系

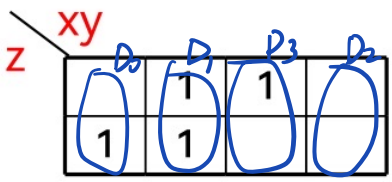
m_i	x y	z	D_i
m_1	0 0	1	$D_0 = z$
m_2	0 1	0	$D_1 = 1$
m_3	0 1	1	
m_6	1 1	0	$D_3 = \bar{z}$

y \ x		
	z	0
	1	$\bar{z}$

但有输出与z无关



② 列出函数F的卡诺图



$D_0 = z$

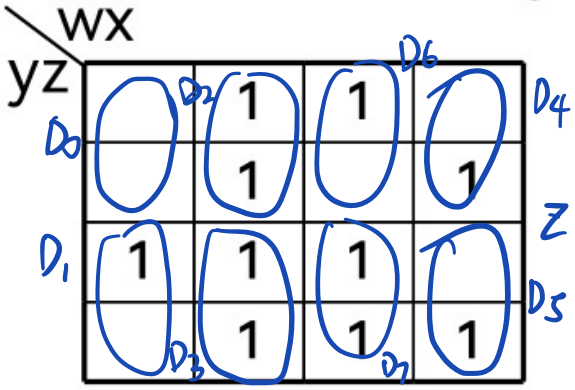
$D_1 = 1$

$D_2 = 0$

$D_3 = \bar{z}$

例3 $F(w,x,y,z) = \Sigma m^4\ (3,4,5,6,7,9,10,12,14,15)$

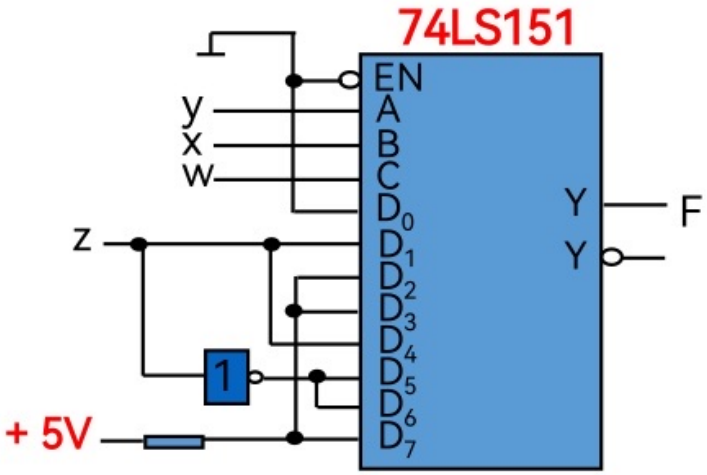
①选择有三个输入选择变量的 8 输入 1 位多路选择器 74LS151。将w、x、y分别接入地址端，z 接入数据端。



$D_0 = 0$ $D_1 = z$ $D_2 = 1$

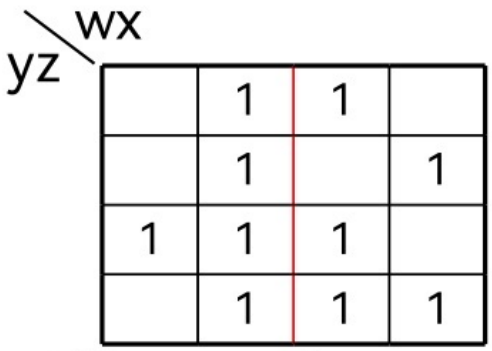
$D_3 = 1$ $D_4 = z$ $D_5 = \overline{z}$

$D_6 = \overline{z}$ $D_7 = 1$



②选择4 输入 2 位多路选择器74LS153。

将w、x、y作为地址端，z 作为数据端。

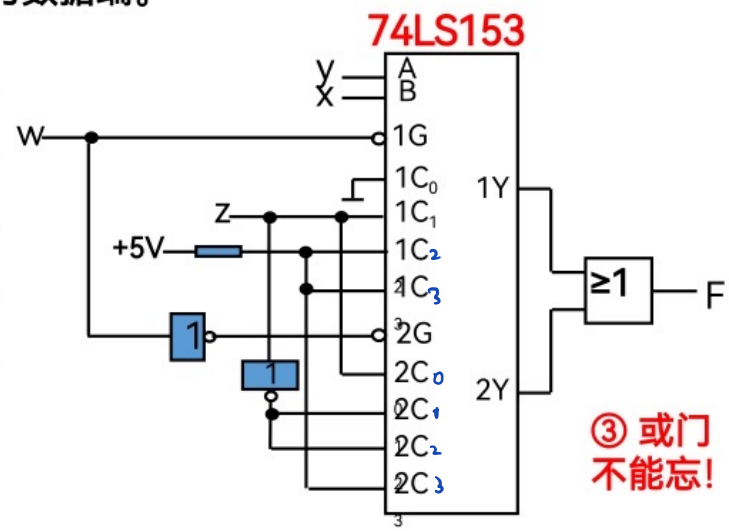


当w=0 时：$D_0 = 0$ $D_1 = z$

$D_2 = 1$ $D_3 = 1$

当w=1 时：$D_4 = z$ $D_5 = \overline{z}$

$D_6 = \overline{z}$ $D_7 = 1$



③ 或门不能忘！

四、比较器和加法器：

1.比较器：

比较器是对**两个位数相同的二进制整数**进行数值比较，并判断其大小关系的逻辑器件。

比较大小关系的结果有三种：

大于(>)、等于(=)、小于(<)

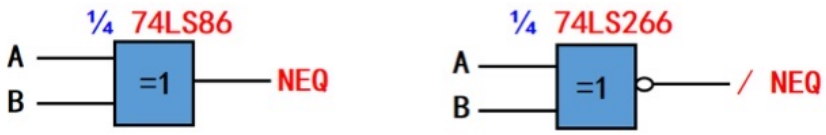
相等的比较过程

相等比较的过程**总是从高位开始比较**，只有当相同位比较结果相等时，才进行低位比较。因此，两个一位二进制数的比较是整个比较器操作的基础。

判断两个一位二进制数是否相等，可用异或门、异或非门(**同或门**)实现，其逻辑表达式为：

$$NEQ = A \oplus B \qquad /NEQ = \overline{A \oplus B} = A \odot B$$

对应的两个逻辑电路如下图所示。



同理，**异或门**及**异或非门**也可以实现两个多位数的比较。

例 四位二进制数的相等比较器。

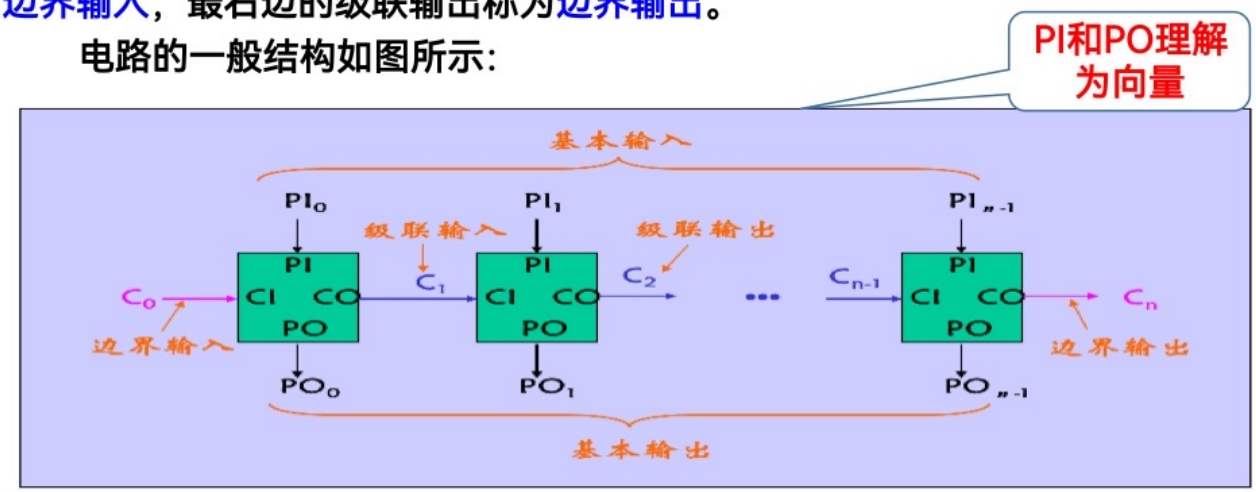
$$NEQ = (A_0 \oplus B_0) + (A_1 \oplus B_1) + (A_2 \oplus B_2) + (A_3 \oplus B_3)$$

(1) 重复电路 (串行组合电路)

串行重复电路是一种组合逻辑电路，它包含几个同样的模块，每个**模块有四种类型**的输入和输出：

基本输入、基本输出、级联输入、级联输出，其中：最左边的级联输入称为**边界输入**，最右边的级联输出称为**边界输出**。

电路的一般结构如图所示：



串行电路非常适用于简单、重复的逻辑运算问题，用串行重复电路可以组成串行比较器、串行加法器和减法器等。

串行电路的重复操作步骤如下：

(1) C_0 置初值，并置 i 为0；
(2) 用C_i 和 PI_i 运算得到 PO_i和 C_{i+1}；
(3) i + 1 → i；
(4) 如果 i 小于 n，返回步骤(2)。

PI和PO理解为向量

在串行重复电路中，通过提供给各个模块基本的输入 PI_0 ~ PI_n ，

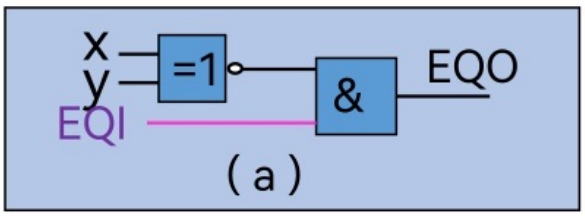
利用模块的串行级联完成步骤(2)~(4)的循环。

例　串行重复比较电路

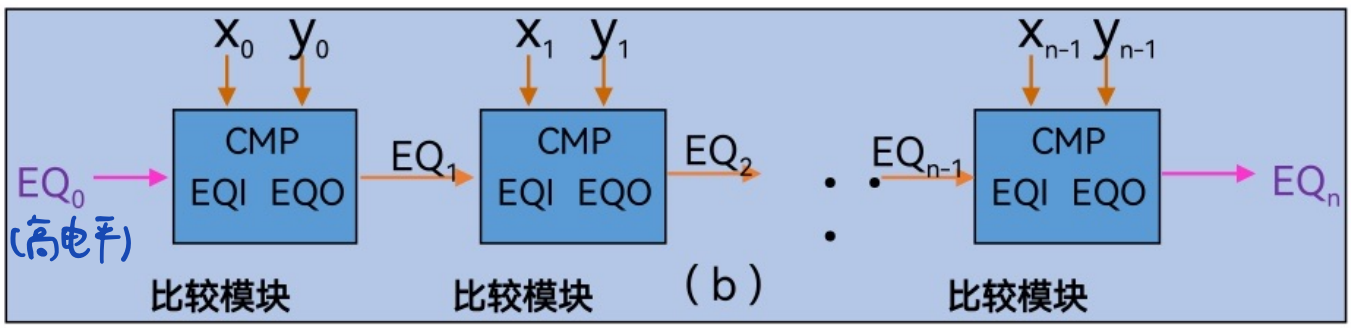
优点？缺点？

判断二个 n 位2进制数是否相等的串行比较电路，如图(b)所示：

➤一位串行比较模块参见图(a)



➤完整的 n 位串行比较电路参见图(b)



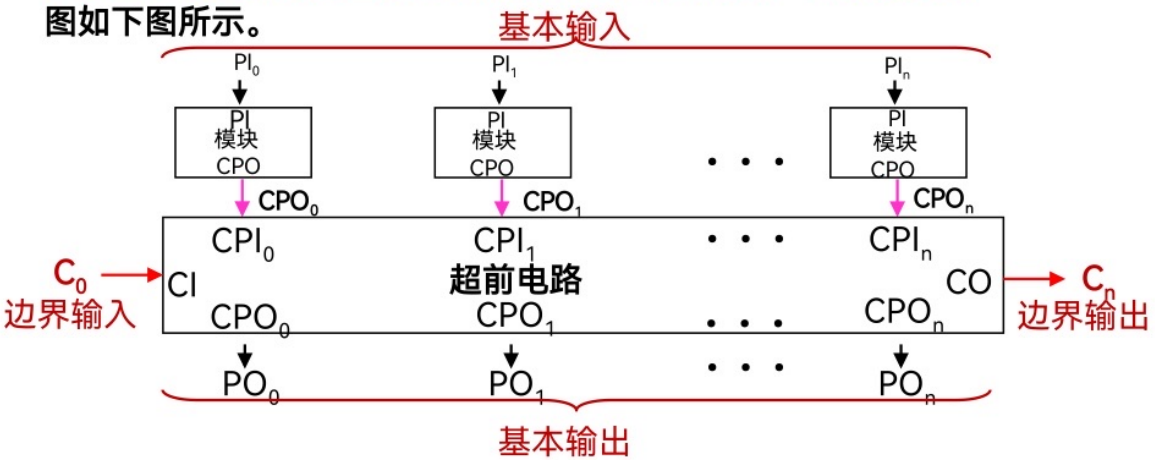
优：无需与或门直接输出

缺：速度慢

(2) 超前电路

与串行电路的区别？→并行，更快

在串行电路中，计算延迟时间随着参与运算的位数n的增加而增大。为了提高速度，采用超前电路。即各个模块直接产生供超前电路进行运算的中间信号，由超前电路对这些信号同时进行处理，从而产生输出结果。超前电路框图如下图所示。



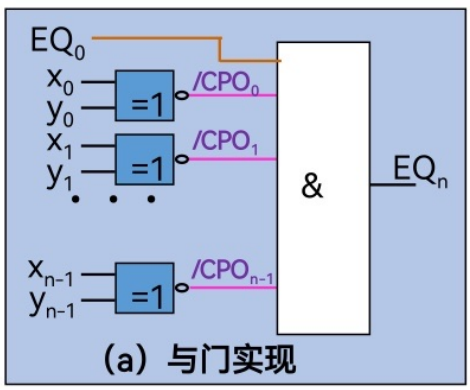
基本输入：x_i 和 y_i
边界输入：EQ_0
边界输出：EQ_n
中间变量：CPO_i
$(i = 0,1,\ldots,n)$

例 超前相等比较器

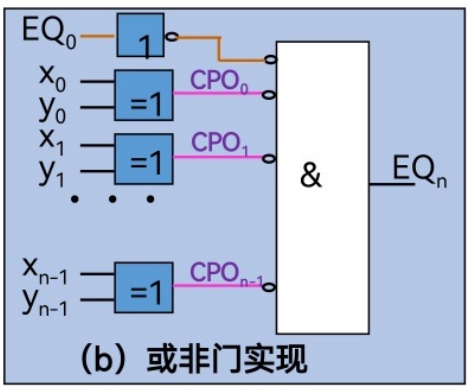
$$CPO_i = x_i \oplus y_i \qquad i = 0,1,\ldots,n-1$$

$$EQ_n = EQ_0 \cdot \overline{CPO_0} \cdot \overline{CPO_1} \cdot \ldots \cdot \overline{CPO_{n-1}} \quad (a)$$

$$= \overline{EQ_0 + CPO_0 + CPO_1 + \ldots + CPO_{n-1}} \quad (b)$$



(a) 与门实现

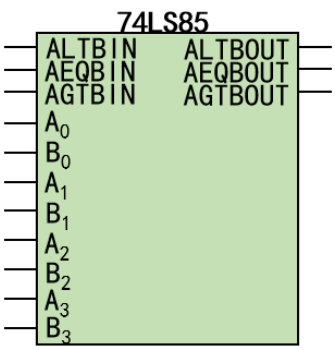


(b) 或非门实现

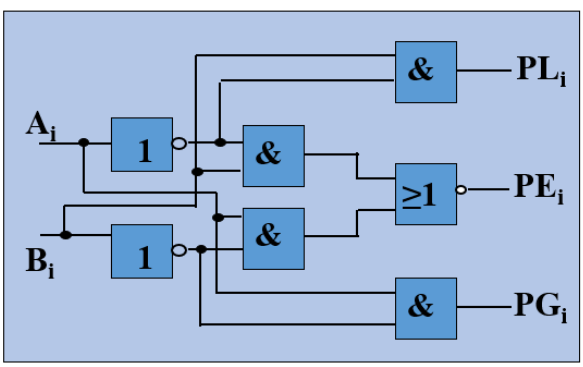
(3) MSI比较器

➢四位比较器74LS85

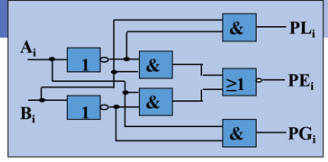
① 电路的逻辑符号



② 一个输入处理模块的逻辑框图



4位数比较时，ALTBIN=AGTBIN=0，AEQBIN=1。



③ 逻辑表达式

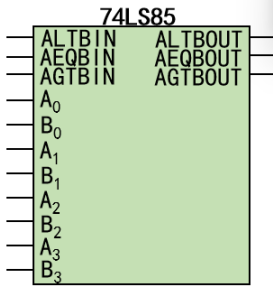
四位二进制比较器有四个输入处理模块，产生了12个中间变量，它们是：

$PG_0=A_0\overline{B_0}$ ； $PE_0=\overline{A_0 \oplus B_0}$ ； $PL_0=\overline{A_0}\,B_0$

$PG_1=A_1\overline{B_1}$ ； $PE_1=\overline{A_1 \oplus B_1}$ ； $PL_1=\overline{A_1}\,B_1$

$PG_2=A_2\overline{B_2}$ ； $PE_2=\overline{A_2 \oplus B_2}$ ； $PL_2=\overline{A_2}\,B_2$

$PG_3=A_3\overline{B_3}$ ； $PE_3=\overline{A_3 \oplus B_3}$ ； $PL_3=\overline{A_3}\,B_3$



如何才能使整个电路的三个输出结果呢？

④ 四位比较器的输出逻辑表达式

$AGTBOUT = (A>B) + (A=B)\cdot AGTBIN$

$= PG_3+PE_3\cdot PG_2+PE_3\cdot PE_2\cdot PG_1+PE_3\cdot PE_2\cdot PE_1\cdot PG_0$

$+PE_3\cdot PE_2\cdot PE_1\cdot PE_0\cdot AGTBIN$

$A_3>B_3$，或
$A_3=B_3$且$A_2>B_2$，或
$A_3A_2=B_3B_2$且$A_1>B_1$，或
$A_3A_2A_1=B_3B_2B_1$且$A_0>B_0$，或
$A_4A_3A_2A_1=B_4B_3B_2B_1$且AGTBIN有效

$AEQBOUT = (A=B)\cdot AEQBIN$

$= PE_3\cdot PE_2\cdot PE_1\cdot PE_0\cdot AEQBIN$

$A_4A_3A_2A_1=B_4B_3B_2B_1$且AEQBIN有效

$ALTBOUT = (A<B) + (A=B)\cdot ALTBIN$

$= PL_3+PE_3\cdot PL_2+PE_3\cdot PE_2\cdot PL_1+PE_3\cdot PE_2\cdot PE_1\cdot PL_0$

$+PE_3\cdot PE_2\cdot PE_1\cdot PE_0\cdot ALTBIN$

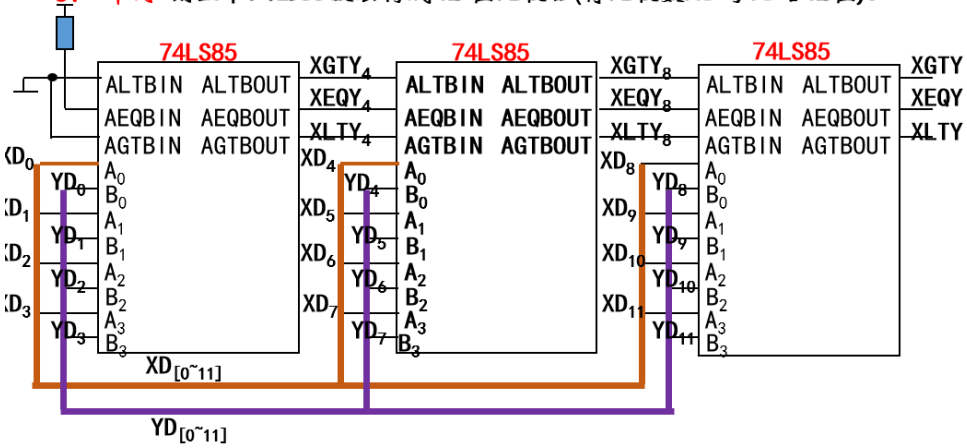
$A_3<B_3$，或
$A_3=B_3$且$A_2<B_2$，或
$A_3A_2=B_3B_2$且$A_1<B_1$，或
$A_3A_2A_1=B_3B_2B_1$且$A_0<B_0$，或
$A_4A_3A_2A_1=B_4B_3B_2B_1$且ALTBIN有效

⑤ 逻辑电路图

74LS85的逻辑符号，参见书P84图2.71

⑥ 74LS85比较器的级联

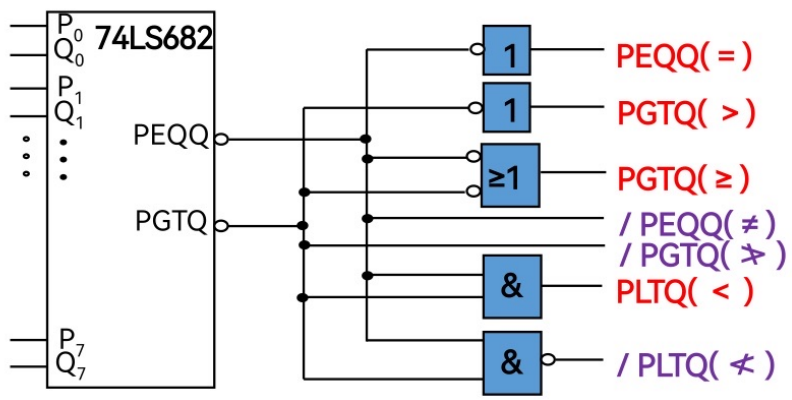
举例 用三个74LS85级联构成12位比较器(待比较数XD与YD各12位)。



➤八位比较器74LS682

74LS682有两个低有效输出端：PEQQ(等于) 及 PGTQ(大于)

逻辑符号及各种条件输出如下图所示。



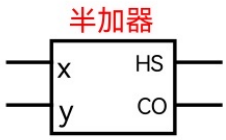
2.加法器：

➤ 半加器的 HS 和 CO 的逻辑表达式为：

$$HS = x \oplus y = x \cdot \overline{y} + \overline{x} \cdot y$$

$$CO = x \cdot y$$

逻辑符号如右图所示。

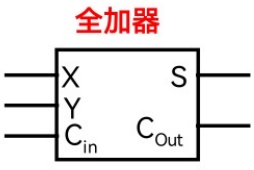


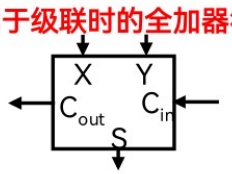
➤ 全加器的 S 和 C_{out} 的逻辑表达式为：

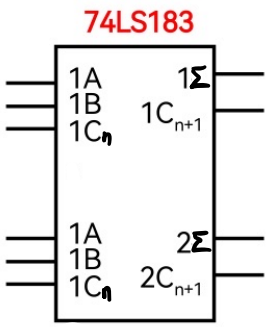
$$S = x \oplus y \oplus C_{in}$$

$$C_{out} = x \cdot y + x \cdot C_{in} + y \cdot C_{in}$$

逻辑符号如下图所示。

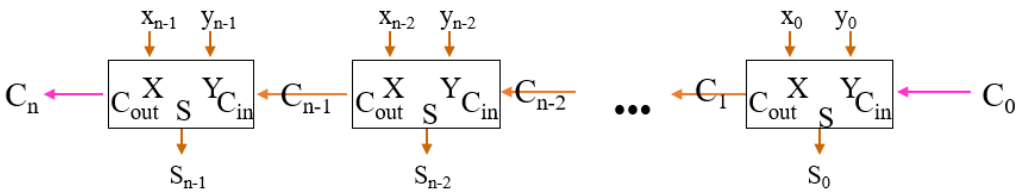






(1) n位加法器（行波加法器,串行加法器）

n个全加器级联，每个全加器处理两个一位二进制数相加，则可以构成两个n位二进制数相加的加法器。由于进位信号是逐级地由低位向高位产生，又称为行波加法器。



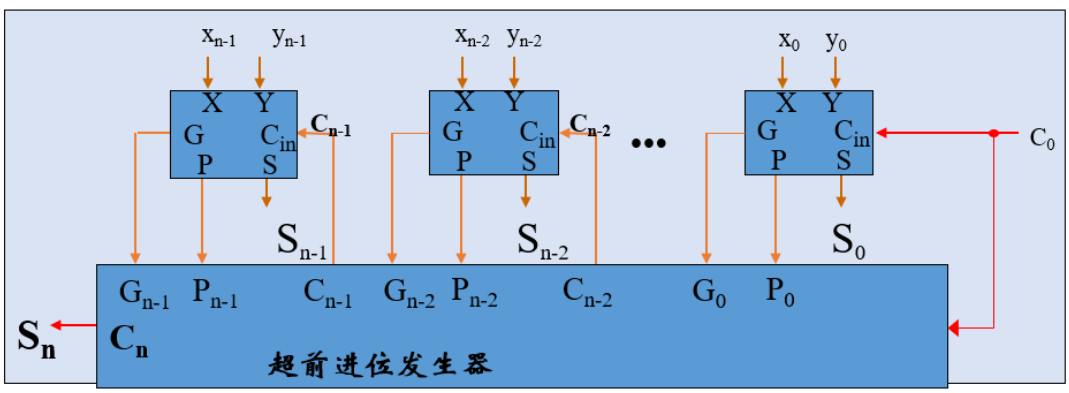
由于进位信号逐位产生，这种加法器速度低。最坏的情况时进位从最低位传送至最高位。行波加法器的最大运算时间为：

$$T_{ADD} = T_{XYCOUT} + (n-2) \cdot T_{CINCOUT} + T_{CINS}$$

其中：T_{XYCOUT} 是最低位全加器中由 x 和 y 产生进位 C_{out} 的延迟时间，$T_{CINCOUT}$ 是中间位全加器中由 C_{in} 产生 C_{out} 的延迟时间，T_{CINS} 是最高位全加器中由 C_{in} 产生 S 的延迟时间。

(2) 超前进位加法器

超前进位加法器的结构框图如下图所示：



输入处理模块的逻辑表达式为：

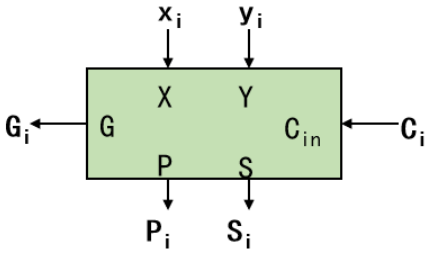
$$S = x \oplus y \oplus C_{in} = P \oplus C_{in}，令 G = x \cdot y，P = x \oplus y$$

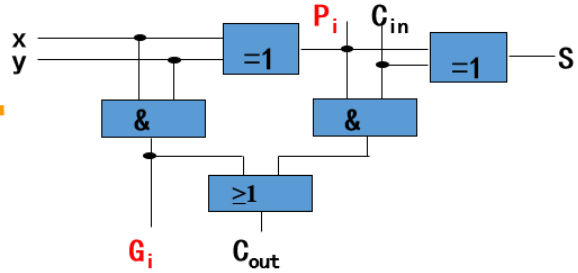
$$C_{out} = x \cdot y + (x \oplus y) C_{in} = G + P \cdot C_{in}$$

如何通过超前电路得到各 C_i？

$G_i = x_i \cdot y_i$， $P_i = x_i \oplus y_i$
$S_i = P_i \oplus C_{in}$
$C_{out} = G_i + P_i \cdot C_{in}$

可以由二个半加器组成的一个全加器构成输入处理模块，如下：





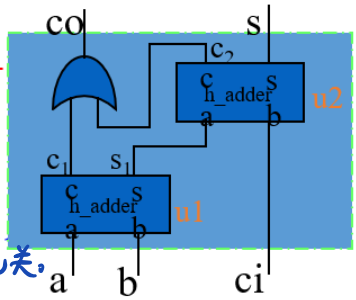
例 三位二进制加法的进位输出可写成：

$C_1 = G_0 + P_0 \cdot C_0$

$C_2 = G_1 + P_1 \cdot C_1 = G_1 + P_1 \cdot (G_0 + P_0 \cdot C_0) = G_1 + P_1 \cdot G_0 + P_1 \cdot P_0 \cdot C_0$

$C_3 = G_2 + P_2 \cdot C_2 = G_2 + P_2 \cdot (G_1 + P_1 \cdot G_0 + P_1 \cdot P_0 \cdot C_0)$
$= G_2 + P_2 \cdot G_1 + P_2 \cdot P_1 \cdot G_0 + P_2 \cdot P_1 \cdot P_0 \cdot C_0$

逻辑图参见教材P87图2.77。



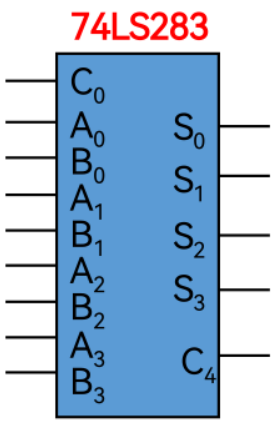
特点：所有进位都是同时产生的，故电路延时时间与位数多少无关，在位数较多时其运算速度比行波加法器的要快得多。

(3) MSI加法器74LS283

这是一个快速进位四位二进制加法器。 逻辑符号如下所示。

可以用74LS283级联构成 n 位加法器，级联方式为：

- 片内（4位）超前进位； 片间为行波进位



例1 用MSI四位二进制加法器实现两个一位十进制8421BCD码的加法器。

- 一位8421BCD码所能表示的值是 0~9;
- 两个8421BCD码相加所得和的范围是0~18;
- 如果其和的范围在 0~9 之间，则不需要进行校正;
- 如果其和在 10~18 之间，则必须进行校正。

参见下页真值表所示。

两个一位8421码相加之和的校正表

何时需要校正？>9时

十进制数	未校正BCD码和 $C_4\ S_3\ S_2\ S_1\ S_0$	校正的BCD码和 $C_4'\ S_3'\ S_2'\ S_1'\ S_0'$	十进制数	未校正BCD码和 $C_4\ S_3\ S_2\ S_1\ S_0$	校正的BCD码和 $C_4'\ S_3'\ S_2'\ S_1'\ S_0'$
0	0 0 0 0	0 0 0 0	10	1 0 1 0	1 0 0 0 0
1	0 0 0 1	0 0 0 1	11	1 0 1 1	1 0 0 0 1
2	0 0 1 0	0 0 1 0	12	1 1 0 0	1 0 0 1 0
3	0 0 1 1	0 0 1 1	13	1 1 0 1	1 0 0 1 1
4	0 1 0 0	0 1 0 0	14	1 1 1 0	1 0 1 0 0
5	0 1 0 1	0 1 0 1	15	1 1 1 1	1 0 1 0 1
6	0 1 1 0	0 1 1 0	16	1 0 0 0 0	1 0 1 1 0
7	0 1 1 1	0 1 1 1	17	1 0 0 0 1	1 0 1 1 1
8	1 0 0 0	1 0 0 0	18	1 0 0 1 0	1 1 0 0 0
9	1 0 0 1	1 0 0 1			

$$C_4' = S_3\overline{S_2}S_1\overline{S_0} + S_3\overline{S_2}S_1S_0 + S_3S_2\overline{S_1}\,\overline{S_0} + S_3S_2\overline{S_1}S_0 + S_3S_2S_1\overline{S_0} + S_3S_2S_1S_0 + C_4$$
$$= S_3S_1 + S_3S_2 + C_4$$

$S_3'\ S_2'\ S_1'\ S_0' = S_3S_2S_1S_0 + 0110$

两个一位8421码加法器的逻辑图

当和需要校正(即C_4'= 1)时，则需作+0110(+6)校正。

$C_4'=S_3S_1 + S_3S_2 + C_4$

$S_3'S_2'S_1'S_0'=S_3S_2S_1S_0 + 0110$

